Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Devices, Interrupts, and System Calls **5**

1.1 Unit 1: Devices and a Polled Driver 7
1.1.1 CMSDK and CMSIS: Hardware versus Software 7
1.1.2 The Emulated Board: MPS2 with AN385 7
1.1.3 Hardware Architecture: AHB vs. APB Buses 8
1.1.4 The Board's Interrupt Lines 8
1.1.5 Software Organization: `include/cmsdk.h` 8
1.1.6 The Board's Memory Map from QEMU 9
Interpreting the Output: Real Models vs. Placeholders 9
Debugging Accesses to Unimplemented Hardware 10
1.1.7 The General I/O Landscape 11
1.1.8 DMA: Moving Data Without the CPU 11
1.1.9 Memory-Mapped I/O: Loads and Stores with Side Effects . . . 12
1.1.10 The `volatile` Keyword 12
Try It: `volatile` at `-O2` 12
1.2 Panoramica del Sistema: Architettura Interna e Comunicazione . . . 14
1.2.1 La Topologia dei Bus: AHB e APB 14
1.2.2 Dalla Simulazione al Bare-Metal: Il Ruolo della UART0 . . . 14
1.3 The Dangers of Read-Modify-Write (RMW) 15
1.3.1 Safer Hardware Patterns 16
1.4 The Device Protocol and the Driver Concept 16
1.4.1 The Hardware State Machine 16
Configuring the Baud Rate 17
1.5 Implementing the Device Driver 18
1.5.1 How Bigger OSes Generalize a Driver 18
1.5.2 Implementing and Using the UART Driver 18
Testing the Driver 19
1.5.3 Try It: A Polled Echo 19
1.6 Variadic Functions and `<stdarg.h>` 20
1.6.1 Separating Formatting from Output (`vformat`) 21
1.6.2 Writing `vformat` 22
Why `0u - (uint32_t) v` and not `-v`? 23
1.7 The Kernel Print Function (`kprintf`) 23
1.7.1 Kernel Mode vs. User Mode Printing 23
1.7.2 Implementing `kprintf` 24
Why `(void) ctx`? 24
How the `va_list` is passed: 24
1.8 Upgrading the Kernel: Fault Handlers That Print 25

1.8.1 The Status Registers (HFSR and CFSR) . . . 25
1.8.2 Implementing the New Handlers . . . 25
1.8.3 Try It: Catching a Divide-By-Zero . . . 26
1.8.4 Practical Note: Project Navigation and Exam Expectations . 26
1.9 Unit 2: Calling Convention and Interrupts . . . 27
1.9.1 Polling versus Interrupts . . . 27
1.9.2 The AAPCS: How C Calls Functions . . . 27
1.9.3 Managing Interrupts from Assembly: The PRIMASK . . . 28
The Nesting Problem . . . 28
Implementing `irq_save` and `irq_restore` in Assembly . . . 28
Testing the API in C . . . 29
Inspecting the AAPCS with GDB . . . 30
1.10 ARM Exception Entry and Return Mechanics . . . 30
1.10.1 The Automatic Hardware State Save (Stacking) . . . 30
1.10.2 Exception Return: `EXC_RETURN` . . . 31
1.10.3 Tail-Chaining and Late Arrival . . . 31
1.11 The Heartbeat of the OS: The SysTick Timer . . . 32
1.11.1 Hardware Preemption . . . 32
1.11.2 The SysTick Hardware Registers . . . 32
1.11.3 Building the SysTick Driver . . . 33
Choosing the Tick: Reload Value and Rate . . . 33
Uptime in Milliseconds and Wraparound Safety . . . 33
`wfi`: An Idle Loop That Sleeps . . . 34
1.12 The Nested Vectored Interrupt Controller (NVIC) . . . 34
1.12.1 The Three Gates of Interrupt Firing . . . 35
1.13 Expanding the Vector Table for External Devices . . . 36
1.14 Concurrency and Shared Data . . . 36
1.14.1 The Problem: Race Conditions . . . 36
1.14.2 The Solution: Critical Sections . . . 37
1.14.3 The Ring Buffer (Circular Queue) . . . 37
1.14.4 Interrupt-Driven Transmission . . . 38
The Producer: `uart_write` . . . 38
The Consumer: `UART0TX_Handler` . . . 39
Initialization: `uart_init` . . . 39
1.14.5 Testing the Driver . . . 40
1.15 Unit 3: System Calls and the Kernel Boundary . . . 41
1.15.1 Recap: Privilege and `svc` . . . 41
1.15.2 The Rule From Now On: The App Asks, The Kernel Does . . 41
Logical vs. Physical Protection . . . 42
1.15.3 The System-Call ABI and Services . . . 42
1.15.4 The Exception Frame and the Journey of a Syscall . . . 43
1.15.5 The Kernel Entry Point: Why It Must Be Assembly . . . 44
1.15.6 Extracting the Service Number and Arguments . . . 45
1.15.7 The Dispatcher and Validating the Result Return . . . 45
Why Write to `frame[0]`? . . . 46
Try It: Your First System Call . . . 46
Finding the Service Number in GDB . . . 46
1.15.8 The Kernel Rules: Where SVC Works . . . 47

1.15.9 Why the Kernel Never Waits 47
1.15.10 User Space Library: Implementing `printf` 47
Handling Partial Writes and Waiting (`write_all`) 48
1.15.11 Implementing `sleep_ms` in User Space 48
1.15.12 The Final System Boot Flow 49
1.15.13 The User Application: `app_main` 50
1.15.14 A Debug Print Inside the Kernel 50
1.15.15 Making the Kernel Wait 51
1.15.16 Privilege and Layers in GDB 52
1.16 Where to Find the Code 53
1.17 A prova di coglione 54
1.17.1 Unità 1 54
1.17.2 Unità 2 60
1.17.3 Unità 3 66

Chapter 1

Devices, Interrupts, and System Calls

Week 3: Devices, interrupts and system calls

At the end of week 2 your program could boot on its own: your `Reset_Handler` set up the stack, copied `.data`, zeroed `.bss`, configured the CPU's fault handling and priorities, and called `main()`. It printed through semihosting, a trap to the debugger host, and it ended with a first taste of protection: `main` dropped privilege, and an `svc` instruction asked privileged code to print on its behalf.

This week you turn that program into something that looks like a small operating system kernel. You stop relying on the host to print, and talk to a real device, the board's serial port, by reading and writing its registers. You write a **device driver** for it, first in the simplest way (the CPU waits for the device), then in the efficient way (the device tells the CPU when it is ready, through an **interrupt**). You add a clock that ticks every 10 ms. And you finish by building a real **system-call interface**: an unprivileged application that prints, reads keys and sleeps, without ever touching a register itself, because every request goes through the kernel.

By the end of the week every layer between an unprivileged `printf` and the serial port's registers has been written by you, and has a name and a file.

The week is organized in three units, and these notes follow the same order:

1. **Devices and a polled driver.** The board's peripherals and how they are connected, memory-mapped I/O and `volatile`, the serial port (UART), what a driver is, and a `printf`-like function for the kernel.

2. **Calling convention and interrupts.** How C functions pass arguments and results (the AAPCS), what the hardware does on exception entry and return, the SysTick timer, the interrupt controller (NVIC), and an interrupt-driven transmitter with a ring buffer.

3. **System calls: the kernel boundary.** A system-call ABI built on `svc`, the kernel entry in assembly, a dispatch table of services, user-side stubs, a user `printf`, and two rules every kernel follows.

Two optional tutorials go with this week. The *C for device drivers* tutorial covers the C techniques used everywhere in this week's code (fixed-width types, bit operations, register `struct` overlays, variadic functions, ring buffers, function pointers)

with small programs you can run on your laptop. The *gdb debugging* tutorial goes further than the commands used in these notes: breakpoints, watchpoints, the call stack and debugging exceptions, all on this week's code.

Where you start and where you end

This week's code lives in two sibling folders, `week-03/base/` and `week-03/solution/`. Each has its own `Makefile` and builds on its own. You start from `base/` and, step by step, you turn it into the program in `solution/`.

The base is not a copy of week 2's result. It is a skeleton designed for this week, with the parts that are long to type and not this week's topic already in place:

- the full `Makefile`, which builds every source folder and has three targets: `run`, `debug`, and `run-trace` (the same as `run` plus a QEMU option you meet in Unit 1);
- the linker script `scripts/mps2_m3.ld`, unchanged from week 2;
- `include/cmsdk.h`, the complete register map of the board's devices, which you read and use instead of typing;
- `boot/startup.s`, with week 2's complete CPU setup and the 16 core slots of the vector table (the device slots are added in Unit 2);
- `kernel/faults.c`, where the four fault handlers are named, silent infinite loops;
- the headers that declare every function of the week (`drivers/uart.h`, `drivers/systick.h`, `kernel/irq.h`, `kernel/kprintf.h`, `kernel/syscall.h`, `lib/format.h`, `lib/printf.h`, `lib/syscalls.h`, `src/app.h`), so that files can call each other from the start;
- in `lib/format.c`, the two helpers that turn a number into text.

Every other source file exists only as a **marker file**: a header comment and one line starting with `/* CLASS:` that says what goes there. Each step in these notes says which marker line you replace with which code. The base's `src/main.c` is privileged and contains an empty `for ( ;; )` loop with three `CLASS:` markers. If you build and run the base, `make run` shows **nothing**: the first visible output of this week is the first byte you send through the serial port yourself.

The finished program in `solution/` prints, when you run it and press `a` and `b` on your keyboard after a second:

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
uptime: 1000 ms
key: a
key: b
uptime: 2000 ms
uptime: 3000 ms
```

The first two lines come from the privileged kernel. Everything after them comes from an unprivileged application that can only *ask* the kernel to do things.

Unless a path in these notes says otherwise, file paths are relative to `week-03/solution/`. The code is fully commented there, line by line, so you can always compare what you typed with the reference. A practical note: QEMU runs with its serial port on the terminal (`-serial stdio`, monitor disabled), so you stop it with `Ctrl-C`.

1.1 Unit 1: Devices and a Polled Driver

1.1.1 CMSDK and CMSIS: Hardware versus Software

In the ARM ecosystem, it is important to distinguish between two similarly named standards:

- **CMSDK (Cortex-M System Design Kit):** This is **hardware**. It comprises ready-made peripheral designs (e.g., APB UART, APB timer, AHB GPIO) that chip designers place alongside the Cortex-M CPU. The registers of these blocks behave identically across all chips that implement them.
- **CMSIS (Cortex Microcontroller Software Interface Standard):** This is **software**. It defines a standardized structure for C header files and helper functions (e.g., `NVIC_EnableIRQ()`, `UART0->DATA`) provided by chip vendors.

Our emulated board uses CMSDK peripherals. However, to understand the bare-metal mechanics, we avoid vendor CMSIS libraries. Instead, our register map (`include/cmsdk.h`) is hand-written in a *CMSIS style* to maintain standard naming conventions while retaining full control over the underlying implementation.

1.1.2 The Emulated Board: MPS2 with AN385

The target hardware is the **MPS2** ARM development board, configured with the **AN385** FPGA image. This image provides a Cortex-M3 CPU surrounded by CMSDK peripherals. QEMU emulates this configuration via the `mps2-an385` machine target.

The entire system is driven by a **25 MHz clock**. All time-dependent peripheral settings (such as UART baud rates and timer periods) are computed based on this central frequency.

The hardware peripherals are mapped to specific addresses within the Cortex-M peripheral memory region (`0x40000000` to `0x5FFFFFFF`):

Address	Device
`0x40000000 / 0x40001000 / 0x40002000`	TIMER0 / TIMER1 / dual timer
`0x40004000 / 0x40005000 / 0x40006000`	UART0 / UART1 / UART2
`0x40008000`	Watchdog
`0x40010000 ...`	GPIO0 ... (general-purpose pins)
`0x40028000`	FPGAIO (board LEDs and buttons)

1.1.3 Hardware Architecture: AHB vs. APB Buses

A bus is the physical set of wires carrying load and store operations between the CPU, memory, and devices. The Cortex-M3 utilizes two distinct buses:

- **AHB (Advanced High-performance Bus):** The main, pipelined bus capable of completing one transfer per clock cycle. High-speed components like RAM, code memory, and the GPIO controller reside here.
- **APB (Advanced Peripheral Bus):** A simpler, slower, and lower-power bus designed for rarely accessed peripheral registers (e.g., UARTs, Timers, Watchdog).

An **AHB-to-APB bridge** connects the two, translating fast AHB accesses into APB accesses. From a software perspective, this routing is transparent: a register is simply a memory address. However, accessing an APB register incurs a slight performance penalty (a few extra cycles) due to bridge latency.

1.1.4 The Board's Interrupt Lines

Peripherals notify the CPU of critical events (e.g., a byte arriving, or a timer reaching zero) by asserting a physical **Interrupt Request (IRQ)** line. In the Cortex-M architecture, device IRQ line n is serviced by vector table slot $16 + n$.

The primary interrupt map for the AN385 board is as follows:

- **IRQ 0 / 1:** UART0 Receive (RX) / Transmit (TX)
- **IRQ 8 / 9:** TIMER0 / TIMER1
- **IRQ 10:** Dual timer

The Watchdog timer bypasses standard IRQ lines and directly raises a **Non-Maskable Interrupt (NMI)** at vector slot 2. For the initial implementation phase, interrupts will remain disabled, and the CPU will rely only on **polling** (asking the devices in a loop whether they are ready).

1.1.5 Software Organization: `include/cmsdk.h`

To interact with the hardware, the provided `include/cmsdk.h` file describes each device as a C `struct` **laid over its registers**. The first field maps to the device's base address, and each subsequent `uint32_t` field sits exactly 4 bytes after the previous one.

For example, the serial port is defined as `CMSDK_UART_TypeDef`:

```
typedef struct
{
    volatile uint32_t DATA;      /* 0x00 read = received byte, write = by
    volatile uint32_t STATE;     /* 0x04 status flags */
    volatile uint32_t CTRL;      /* 0x08 control flags */
    volatile uint32_t INTSTATUS; /* 0x0C interrupt status; write 1 to cle
    volatile uint32_t BAUDDIV;   /* 0x10 baud-rate divider */
} CMSDK_UART_TypeDef;

#define CMSDK_UART0    ( ( CMSDK_UART_TypeDef * ) 0x40004000UL )
```

`CMSDK_UART0` is not a variable; it is the raw memory address `0x40004000` cast to a pointer of this struct type. Therefore, evaluating `CMSDK_UART0->STATE` translates into a 32-bit memory load from address `0x40004004` (the base address plus the offset of the second field).

To ensure architectural safety:

- Every field is declared `volatile` (Section **??** explains why: it stops the compiler from optimizing away hardware reads and writes).
- The file defines specific bit names for each register (e.g., `UART_STATE_TXBF` as `1UL << 0`).
- A `_Static_assert` using `offsetof` is placed at the end of each struct definition to verify the field offsets at compile time, turning any layout typo into an immediate build error.

This single header provides the structured mappings for the SysTick, NVIC, SCB (System Control Block), timers, watchdog, GPIO, and FPGAIO.

1.1.6 The Board's Memory Map from QEMU

Instead of relying solely on documentation, we can ask QEMU directly which hardware components it actually models. QEMU features a **monitor** console to control the emulator.

The most efficient way to extract the hardware layout is by launching QEMU paused (`-S`) with the monitor enabled, passing the `info mtree -f` command, and filtering the massive output for the `0x400xxxxx` peripheral memory region.

We can do this in a single terminal command:

```
printf 'info mtree -f\nquit\n' | qemu-system-arm -M mps2-an385 \
    -display none -S -monitor stdio -serial null \
    | grep ' 00000000400' | sort -u
```

This reveals the active memory map for the board's peripherals:

```
0000000040000000-0000000040000fff (prio 0, i/o): cmsdk-apb-timer
0000000040002000-0000000040002fff (prio 0, i/o): cmsdk-apb-dualtimer
0000000040004000-0000000040004fff (prio 0, i/o): uart
0000000040008000-0000000040008fff (prio 0, i/o): cmsdk-apb-watchdog
0000000040010000-0000000040010fff (prio -1000, i/o): cmsdk-ahb-gpio
0000000040020000-0000000040020fff (prio 0, i/o): pl022
0000000040022000-0000000040022fff (prio 0, i/o): arm_sbcon_i2c
0000000040028000-0000000040028fff (prio 0, i/o): mps2-fpgaio
```

Interpreting the Output: Real Models vs. Placeholders

The `prio` field in the output tells us exactly how QEMU handles each memory range:

- `prio 0` **(Fully Emulated):** Devices like `uart` and `mps2-fpgaio` are real models. For instance, `UART0` at `0x40004000` is fully functional and connected to your terminal via the `-serial stdio` flag.

- `prio -1000` **(Placeholders):** Devices like `cmsdk-ahb-gpio` are "unimplemented". To prevent software crashes, QEMU safely accepts memory accesses here: it silently ignores all writes and returns `0` on all reads, but the hardware itself does not function.

Debugging Accesses to Unimplemented Hardware

If your code isn't working as expected, you might be writing to a placeholder or a non-existent device. You can track this by monitoring QEMU's behavior while manually poking the system bus with GDB.

To see this in action, you need two terminal windows:

- **Terminal 1 (QEMU):** Launch the emulator paused, with the debugger port open and unimplemented device logging enabled (`-d unimp`).
- **Terminal 2 (GDB):** Launch `arm-none-eabi-gdb`, connect to QEMU (`target remote :1234`), and issue memory writes.

Once connected, try writing to a placeholder device (like GPIO) and then reading the value back:

```
(gdb) set {unsigned int}0x40010010 = 1        # 1. Write to GPIO0 (Placeho
(gdb) x/wx 0x40010010                         # 2. Read it back
```

Because GPIO is not implemented, GDB will show the value is `0`. If you look at **Terminal 1**, QEMU explicitly warns you about the ignored access:

```
cmsdk-ahb-gpio: unimplemented device write (size 4, offset 0x010, value 0x00000001)
cmsdk-ahb-gpio: unimplemented device read  (size 4, offset 0x010)
```

Conversely, if you write to a real, fully emulated device like the FPGAIO register:

```
(gdb) set {unsigned int}0x40028000 = 3        # 1. Write to FPGAIO (Real M
(gdb) x/wx 0x40028000                         # 2. Read it back
```

GDB will confirm the value `0x00000003` was successfully retained, and QEMU will remain completely silent.

Summary of Hardware Access Behaviors in QEMU: When your software accesses a peripheral memory address, QEMU resolves it in one of three ways (none of which cause a hardware fault):

1. **Real Model** (e.g., UART0): Works normally. The data is saved, and no warnings are logged.
2. **Device Placeholder** (e.g., GPIO0): Ignored/Zeroed. QEMU logs the specific device name (e.g., `cmsdk-ahb-gpio`) if `-d unimp` is active.
3. **Empty Address** (e.g., 0x40080000): Ignored/Zeroed. QEMU logs a generic region warning (e.g., `RESERVED 4`) if `-d unimp` is active.

Note: Always distinguish between core CPU peripherals (like SysTick, guaranteed by the ARMv7-M architecture) and board-specific peripherals (like UART/GPIO, which depend on the exact emulator model).

1.1.7 The General I/O Landscape

While this course focuses primarily on driving the UART, a modern microcontroller interacts with the outside world through a variety of standardized hardware interfaces:

Interface	Wires	Mechanism	Typical Use
GPIO	1 per pin	Programmable as input (read logic level) or output (drive 0 or 1).	Buttons, LEDs
UART	2 (TX, RX)	Asynchronous: no shared clock. Devices must agree on a transmission speed (baud rate).	Console, PC link
SPI	CLK, Data In, Data Out, CS	Synchronous: controller drives a shared clock. Uses Chip-Select (CS) to pick a specific device.	Flash memory, displays
I2C	2 (CLK, DATA)	Synchronous: shared bus. Controller routes data using 7-bit device addresses.	Sensors
ADC / DAC	1 per channel	Converts an analog voltage to a digital number (ADC) or vice versa (DAC).	Temperature, Audio

Complex interfaces like SPI and I2C are programmed using the same memory-mapped register concepts that we will apply to the UART, simply requiring more state-machine steps per transfer.

1.1.8 DMA: Moving Data Without the CPU

In all the code of this week, the CPU moves every byte itself: it reads a register, stores the value in memory, and repeats. For a large transfer, such as a flash sector or an audio buffer, the CPU is busy for the whole time.

A **DMA controller** (Direct Memory Access) is a device that copies data by itself. You program a source address, a destination address and a count, and start it. The DMA controller then uses the bus while the CPU runs other code, and signals the CPU when it is done, usually with an interrupt. Typical uses are peripheral to memory (bytes received by a UART, samples from an ADC) and memory to peripheral (data to send on SPI). The cost is that the CPU and the DMA controller share the bus, and the software must not touch a buffer while the DMA controller is still filling it. QEMU's AN385 has no DMA controller, which is why the CPU moves every byte here.

1.1.9 Memory-Mapped I/O: Loads and Stores with Side Effects

Every peripheral register has a fixed address, and you talk to a device with ordinary load and store instructions at that address. This is called **memory-mapped I/O** (MMIO). Written without the `struct`, sending a byte looks like this:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000u)
UART0_DATA = 'A';        /* a store to 0x40004000: the UART sends 'A' */
```

`CMSDK_UART0->DATA = 'A';` is exactly the same store, written through the `struct`. It looks like an assignment to a variable, but a device register is **not** RAM, and four differences matter:

- a **read** can have an effect: reading the UART's `DATA` register takes the received byte out of the device;
- a **write** can start an action: writing `DATA` starts sending a byte;
- a register can **change without any store** from the CPU: a status bit goes to 1 when a byte arrives;
- the value you read back may differ from the value you wrote: some bits are read-only, others clear themselves.

The device's manual says what each read and each write does, and your code must follow it exactly.

1.1.10 The `volatile` Keyword

The compiler assumes that it is working with ordinary memory, where a value changes only when **your program** stores to it. With optimization turned on, it keeps values in CPU registers and removes reads it believes are repeated. Consider the loop that waits for the UART to have room:

```
while ( UART0_STATE & UART_STATE_TXBF )    /* wait until the UART has room
{
}
```

Without `volatile`, the compiler sees a loop that never writes `UART0_STATE`. In its model the value "cannot change", so it reads it once and, if the bit was 1, spins forever without looking again.

The keyword `volatile` tells the compiler: **every read and every write in the source must really happen**, in the order written with respect to other `volatile` accesses. You used it in week 2's divide-by-zero demo, to stop the compiler from computing the result at build time. Two things `volatile` does **not** do: it does not make an access atomic (indivisible), and it does not protect data shared with other code. It only forbids the compiler from removing or merging accesses.

Try It: `volatile` at `-O2`

You can watch the compiler delete a hardware read. In a folder outside `base/` (for example `/tmp/vol`), create `poll.c`:

```
#include <stdint.h>

#ifndef QUAL
#define QUAL     /* empty: a plain pointer */
#endif

/* UART0 STATE, bit 0 = TXBF */
#define UART0_STATE (*(QUAL uint32_t *)0x40004004u)

void wait_tx_room(void)
{
    while (UART0_STATE & 1u)
    {
        /* wait until TXBF is 0 */
    }
}
```

Compile it twice with optimization level `-O2`, once plain and once with `-DQUAL=volatile` (which defines `QUAL` as `volatile`), then disassemble both object files:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -O2 -c poll.c -o plain.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -O2 -DQUAL=volatile -c poll.c -
arm-none-eabi-objdump -d plain.o
arm-none-eabi-objdump -d vol.o
```

The question is: in which version does the loop jump back to the `ldr`, the instruction that reads `STATE`? In `plain.o` the read happens once, and the "loop" is a branch to itself:

```
0: mov.w r3, #0x40004000   /* r3 = 0x40004000                          */
4: ldr   r3, [r3, #4]      /* r3 = STATE   (read ONCE)                  */
6: lsls  r3, r3, #31       /* shift bit 0 into the sign: N flag = TXBF */
8: bmi.n c                 /* if (TXBF) goto c                          */
a: bx    lr                /* return                                    */
c: b.n   c                 /* c: goto c  -> for (;;) { }, never reads again */
```

In `vol.o` the loop goes back to the `ldr` and reads `STATE` on every turn:

```
0: mov.w r2, #0x40004000   /* r2 = 0x40004000                 */
4: ldr   r3, [r2, #4]      /* r3 = STATE   (read every turn)  */
6: lsls  r3, r3, #31       /* N flag = TXBF                   */
8: bmi.n 4                 /* if (TXBF) goto 4: read again    */
a: bx    lr                /* return                          */
```

At `-O0`, the level this course's `Makefile` uses, even the plain version happens to re-read the register, so the bug stays hidden until someone turns optimization on. The rule is simple: always access device registers through `volatile`.

1.2 Panoramica del Sistema: Architettura Interna e Comunicazione

Per comprendere come un sistema operativo interagisce con il mondo esterno, è fondamentale visualizzare come sono disposti i componenti hardware all'interno del microcontrollore e come comunicano tra loro.

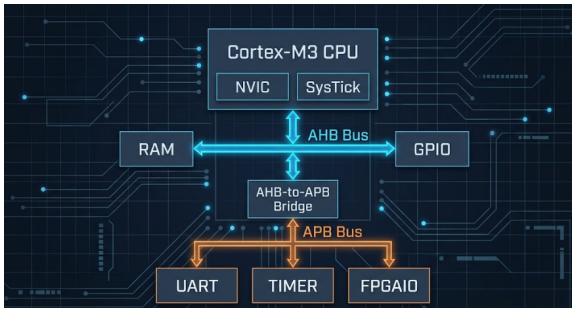


Figure 1.1: Panoramica funzionamento BUS

1.2.1 La Topologia dei Bus: AHB e APB

A livello hardware, il sistema non è piatto: i componenti sono distribuiti su reti diverse (chiamate *Bus*) in base alla velocità e all'energia richiesta.

- **Il Cuore della CPU (Core Peripherals):** Componenti vitali come il **SysTick** (il timer di sistema) e il **NVIC** (il gestore degli interrupt) sono integrati direttamente nel processore Cortex-M3. La CPU vi accede istantaneamente per gestire le emergenze, senza passare per reti esterne.
- **Bus ad Alta Velocità (Bus AHB):** L'*Advanced High-performance Bus* è la rete principale del sistema, capace di scambiare dati alla massima velocità. Viene utilizzata per collegare la **RAM** (memoria) e il controller **GPIO**.
- **La Rete Secondaria (Bus APB):** L'*Advanced Peripheral Bus* è una rete più lenta e a basso consumo energetico. Ospita i dispositivi esterni come la porta **UART** e i **Timer**. Per permettere alla CPU di parlare con questa rete, i dati viaggiano sul bus AHB e attraversano un traduttore hardware chiamato **AHB-to-APB Bridge**.

1.2.2 Dalla Simulazione al Bare-Metal: Il Ruolo della UART0

Nelle settimane precedenti, il nostro codice stampava messaggi a schermo usando il **semihosting**. Questa tecnica è di fatto una "scorciatoia": il processore si bloccava e chiedeva al computer esterno (tramite il debugger) di stampare il testo al posto suo.

Un vero sistema operativo *bare-metal* (indipendente e a basso livello) deve però gestire l'hardware da solo. Per farlo, utilizziamo l'**UART** (*Universal Asynchronous Receiver-Transmitter*), ovvero la porta seriale del microcontrollore, dotata di un canale di trasmissione (**TX**) e uno di ricezione (**RX**).

Perché comunichiamo tramite la UART0?
La scheda emulata possiede diverse porte seriali, ma noi utilizziamo l'`UART0` all'indirizzo `0x40004000` per un motivo pratico legato al nostro emulatore. Quando avviamo QEMU tramite il comando `make run`, utilizziamo il flag `-serial stdio`. Questo comando dice a QEMU di prendere i cavi virtuali TX e RX dell'`UART0` e collegarli direttamente alla nostra finestra del terminale. Di conseguenza:

- Quando il nostro codice C scrive un byte (es. la lettera 'A') nel registro `DATA` dell'`UART0`, l'hardware lo invia sul cavo TX e QEMU lo stampa a schermo.
- Quando premiamo un tasto sulla tastiera, QEMU inietta quel carattere sul cavo RX, permettendo al nostro software di leggerlo interrogando lo stesso registro.

1.3 The Dangers of Read-Modify-Write (RMW)

When configuring a hardware device, we often need to change a single bit while leaving the rest of the register untouched. In C, it is natural to use compound operators like `|=` (e.g., `CMSDK_GPIO0->DATAOUT |= (1u << 3)`).

Even though this looks like a single, instant operation in your code, the processor is forced to execute three distinct steps at the hardware level:

1. **Read:** It loads the current value of the register from memory into the CPU.
2. **Modify:** It mathematically alters the target bit using a bitwise operation.
3. **Write:** It saves the new value back, overwriting the hardware register.

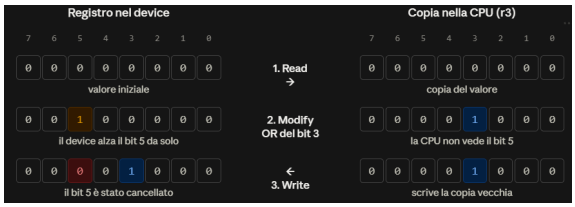


Figure 1.2: Read - Modify - Write

This sequence is **not atomic**, meaning it can be interrupted halfway through. This introduces three classic pitfalls when writing device drivers:

- **Concurrency conflicts:** If an interrupt fires (or the hardware autonomously changes another bit) between your Read and Write phases, your final Write will forcefully overwrite the entire register with the outdated data you initially read, effectively erasing the hardware's recent changes.
- **The W1C (Write-1-to-Clear) Trap:** Many status registers turn off an alarm (flag) when you write a `1` to them. If you use an RMW pattern (`|=`) here, you will read back all the currently active alarms (as `1`s) and write them straight back, accidentally resetting every pending interrupt flag at once.
- **Read/Write Asymmetry:** In some registers, what you read means something entirely different from what you write. A Read-Modify-Write cycle on these registers will scramble the bits unpredictably.

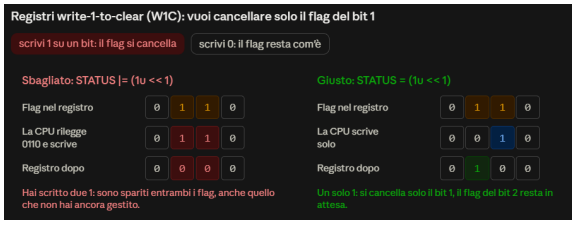


Figure 1.3: Write 1 to Clean Trap

1.3.1 Safer Hardware Patterns

To communicate with registers safely, adopt these practices:

- **Use Dedicated Registers (Set/Clear):** Modern peripherals offer special registers (like `OUTENABLESET`) designed to be used with a simple direct assignment (`=`). The hardware handles it internally, ensuring only the bits set to `1` are modified in a single, safe operation.
- **How to Handle W1C:** To clear a flag in a W1C register, **always** use a direct assignment (`=`) targeting only the specific bit you want to clear. **Never** use `|=`.
- **Careful with C Bit-Shifts:** Always write `1u << n` rather than `1 << n`. The `u` forces the compiler to treat the number as unsigned. Shifting a signed `1` into the 31st bit is undefined behavior in C and will cause glitches.



Figure 1.4: Read - Modify - Write Solutions

1.4 The Device Protocol and the Driver Concept

A **device driver** is a dedicated piece of software that acts as a translator between the operating system and the physical hardware. Instead of forcing the main application to memorize memory addresses and bitwise operations, the driver hides this complexity behind a clean API boundary.

It offers the rest of the system **functions with a clear meaning** (e.g., `uart_putc('A')` to "send this byte"), while internally dealing with the strict hardware rules. Callers simply include the driver's header (`drivers/uart.h`) and never touch the raw register map (`cmsdk.h`).

1.4.1 The Hardware State Machine

Interacting with a hardware device is never as simple as writing to a single variable. It requires following a strict state machine protocol dictated by the device's manual.

A device is a **protocol** expressed through its registers, always executed in a fixed, logical sequence:

1. **Configure** (e.g., `BAUDDIV`): Set up the operating parameters before turning anything on. For a UART, this means calculating and setting the transmission speed (baud rate).

2. **Enable** (e.g., `CTRL`): Power on the specific sub-components (like the transmitter or receiver). You must enable the device *only after* it has been fully configured.

3. **Check Status** (e.g., `STATE`): Never blindly send or receive. You must read the status flags to ensure the hardware buffer actually has room to accept a new byte, or that a new incoming byte is truly waiting to be read.

4. **Move Data** (e.g., `DATA`): Once the status is confirmed to be clear and ready, you can safely read or write the actual data payload.

5. **Acknowledge** (e.g., `INTSTATUS`): If the device generated an event (such as triggering an interrupt), you must clear its specific flag to tell the hardware you have handled the event, allowing it to report the next one.

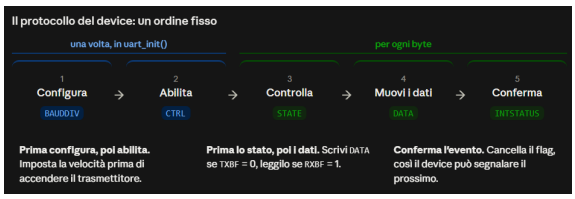


Figure 1.5: Device Driver Protocol - steps

Configuring the Baud Rate

Because UART is asynchronous, both devices must agree on a transmission speed (baud rate). You divide your CPU's clock frequency by your target baud rate. If our CPU runs at 25 MHz and we want 115,200 baud:

$$\text{BAUDDIV} = \frac{25,000,000}{115,200} \approx 217$$

Writing 217 into `BAUDDIV` configures the hardware correctly.

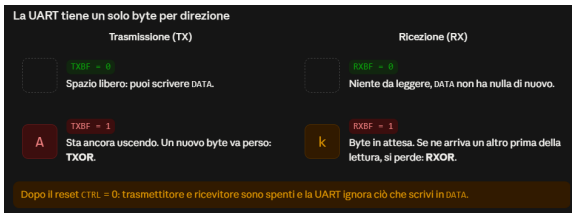


Figure 1.6: How transmission flags works

1.5 Implementing the Device Driver

1.5.1 How Bigger OSes Generalize a Driver

If a system has many character devices, the OS needs a generic way to talk to them. Operating systems solve this by defining a `struct` composed of function pointers:

```
struct char_driver
{
    void (*init)(void);          /* bring the device up */
    int  (*read)(void);          /* read one byte, or -1 if none */
    void (*write)(char c);       /* send one byte */
};

static const struct char_driver uart0_driver = {
    uart_init,
    uart_getc_polled,
    uart_putc_polled
};
```

This allows generic code to interact with any device (e.g., `dev->write('h');`) without knowing its specifics. Linux does exactly this with `struct file_operations`. This week's kernel has a single console and calls the UART functions directly, so you will not write this `struct` yet.

1.5.2 Implementing and Using the UART Driver

In `drivers/uart.c`, paste the following right **above** the marker line `/* CLASS: uart_init, polled putc/getc, ...` (keep the marker for Unit 2).

```
#include "drivers/uart.h"
#include "cmsdk.h"

void uart_init(void)
{
    /* configure: speed (QEMU ignores it) */
    CMSDK_UART0->BAUDDIV = 16;
    /* enable: transmitter and receiver */
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;
}

void uart_putc_polled(char c)
{
    /* check status: wait for room */
    while (CMSDK_UART0->STATE & UART_STATE_TXBF)
    {
    }
    /* move data: send the byte */
    CMSDK_UART0->DATA = (uint32_t)(unsigned char)c;
}
```

Following the protocol, `uart_init` configures and enables the hardware. Because the UART is an external physical device, initialization happens only **once** at boot. We combine two bitmasks with a bitwise OR to enable both directions simultaneously.

In `uart_putc_polled`, if the `TXBF` bit is 1, the CPU loops waiting; once free, the byte is stored into `DATA`. **Why the double cast?** On Arm, `char` is unsigned, but on many compilers it is signed. A signed `char` holding `0xE9` is -23, which converts to `uint32_t` as `0xFFFFFFE9`. Casting through `unsigned char` ensures it becomes `0x000000E9`, extracting exactly the byte.

Testing the Driver In `src/main.c`, add `#include "drivers/uart.h"` above `main`, and right after the marker `/* CLASS: bring up the UART here */` add:

```
#include "drivers/uart.h"

int main(void)
{
    /* CLASS: bring up the UART here */
    uart_init();
    uart_putc_polled('O');
    uart_putc_polled('K');
    uart_putc_polled('\n');
    for (;;) { }
}
```

`make run` prints `OK`: the first real hardware output of our OS. If you comment out `uart_init()`, `CTRL` remains 0 and the UART drops every byte. Running QEMU with `-d guest_errors` reports:

```
CMSDK APB UART: transmit data write with Tx disabled
```

This enforces the rule "enable before moving data".

1.5.3 Try It: A Polled Echo

In `drivers/uart.c`, right after `uart_putc_polled`, add the reading function. It returns `-1` immediately if no character is waiting:

```
int uart_getc_polled(void)
{
    /* check status: nothing received yet */
    if ((CMSDK_UART0->STATE & UART_STATE_RXBF) == 0)
    {
        return -1;
    }
    /* move data: reading DATA takes the byte */
    return (int)(CMSDK_UART0->DATA & 0xFF);
}
```

In `src/main.c`, replace the empty `for (;;) { }` at the end of `main` with an echo loop:

```
for (;;)
{
    int c = uart_getc_polled();
    if (c >= 0)              /* -1: no key */
    {
        uart_putc_polled((char)c);
    }
}
```

Run `make run` and type `hello`. With `-nographic`, the terminal does not show what you type: every letter you see was **sent back by your program**. The Enter key sends a carriage return (`\r`), jumping the cursor back to the start of the line. The CPU spins in this loop millions of times per second: this is the price of polling.

1.6 Variadic Functions and <stdarg.h>

Currently, our driver can only print single characters. To build a true operating system, we need a `printf`-like function capable of formatting complex strings and numbers (e.g., `%d`, `%s`, `%x`). We will call our kernel-level version `kprintf`.

To build `kprintf`, we face two tasks:

1. String formatting (parsing the `%d` markers and converting integers to ASCII characters).
2. Sending the resulting characters to the UART.

In C, a function that accepts an arbitrary number of arguments (like `printf`) is called a **variadic function**. It is declared using an ellipsis (`...`):

```
void kprintf(const char *fmt, ...);
```

`<stdarg.h>` is one of the headers a freestanding build still gives you, as you saw in Week 2: it contains only types and compiler built-ins, no library code. Behind the scenes, it provides macros to navigate the memory (the stack or registers) where the extra arguments are stored. Here is exactly what the macros do:

- `va_list ap`: Think of this as an "argument pointer" (hence `ap`). It is a special cursor variable that keeps track of your current position within the hidden list of extra arguments.
- `va_start(ap, fmt)`: This initializes the cursor. By passing the last known fixed argument (`fmt`), the compiler calculates exactly where in memory the variable arguments begin.
- `va_arg(ap, int)`: This macro performs two actions simultaneously. First, it reads the value at the cursor's current position (interpreting it as the specified type, like `int`). Second, it automatically advances the cursor by the exact number of bytes required to reach the next argument.
- `va_end(ap)`: This cleans up the cursor. While on some architectures (like our Cortex-M3) it might do nothing behind the scenes, it is strictly mandatory for C standard compliance and prevents memory corruption on more complex systems.

```
#include <stdarg.h>

void kprintf(const char *fmt, ...)
{
    va_list ap;                 /* 1. create the cursor */
    va_start(ap, fmt);          /* 2. point it right after 'fmt' */
    int n = va_arg(ap, int);    /* 3. read the first extra arg as int, a
    va_end(ap);                 /* 4. clean up */
}
```

A variadic function cannot see how many arguments it received, nor their types. The **format string** tells it: `%d` means "take an `int`". If the type in the format and the type actually passed disagree, the behavior is undefined. Arguments passed through `...` are also **promoted**: a `char` arrives as an `int`, so `%c` must read it with `va_arg(ap, int)`.

By convention, a function that receives a `va_list` instead of `...` has a name starting with `v`, such as `vprintf` in the C library. The *C for device drivers* tutorial (Section 4) shows variadic functions, promotions and a tiny format dispatcher you can run on your laptop.

1.6.1 Separating Formatting from Output (`vformat`)

Turning `"x=%d"` and `42` into `x=42` is the same work wherever the characters end up. So `lib/format.h` (given) separates **what** to print from **where** it goes:

To maintain a clean operating system architecture, we separate the formatting engine from the hardware driver using a function called `vformat` (implemented in `lib/format.c`).

```
void vformat(void (*out)(char c, void *ctx), void *ctx,
             const char *fmt, va_list ap);
```

The `out` parameter is a **function pointer**. Instead of hardcoding `uart_putc_polled` directly inside the formatting loop, we pass the output function as a variable. `vformat` calls `out(c, ctx)` once for every character it produces. This decouples the logic: `vformat` formats text, completely unaware of whether the characters are going to a UART, a screen, or a network socket.

The `void *ctx` (context) is an **opaque pointer**. This is a standard C design pattern used to pass arbitrary state data through a generic function. `vformat` does not know or care what `ctx` points to; it simply receives it and hands it untouched to your `out` function. If your output function needs to keep track of a string buffer or a character counter, you pass a pointer to that structure as `ctx`.

`kprintf` is then built by calling `vformat` with an `out` function that calls `uart_putc_polled`. Since a UART just blasts characters out and requires no memory state tracking, we simply pass `0` (or `NULL`) as the `ctx`.

Another destination needs only another small `out` function: the formatting is written and tested once.

`vformat` supports `%c`, `%s`, `%d` (signed decimal), `%u` (unsigned decimal), `%x` (hexadecimal, lowercase, without `0x`) and `%%` (a percent sign).

There is no width and no precision, and an unknown conversion such as `%q` is printed as written.

The two helpers that turn a number into text, `utoa_dec` and `utoa_hex`, are already in `lib/format.c` in the base.

1.6.2 Writing `vformat`

In `lib/format.c`, replace the marker line `/* CLASS: vformat (the % dispatch: %c %s %d %u %x %%) goes here. */` with a helper and the function itself.

The helper, `put_string`, sends every character of a string to `out`:

```
static void put_string(void (*out)(char c, void *ctx), void *ctx,
                       const char *s)
{
    while (*s != '\0')
    {
        out(*s, ctx);
        s++;
    }
}
```

`vformat` walks through the format string. An ordinary character is passed on; a `%` makes it look at the next letter and take one argument of the right type:

```
void vformat(void (*out)(char c, void *ctx), void *ctx,
             const char *fmt, va_list ap)
{
    char num[12];    /* longest number + '\0' */

    while (*fmt != '\0')
    {
        if (*fmt != '%')                /* ordinary character: pass it on
        {
            out(*fmt, ctx);
            fmt++;
            continue;
        }
        fmt++;                          /* a '%': look at the next letter
        switch (*fmt)
        {
        case 'c':
            out((char)va_arg(ap, int), ctx);    /* a char arrives as an in
            break;
        case 's':
        {
            const char *s = va_arg(ap, const char *);
            put_string(out, ctx, s != 0 ? s : "(null)");  /* no crash on
            break;
        }
        case 'd':
        {
            int v = va_arg(ap, int);
```

```
            uint32_t mag;
            if (v < 0) { out('-', ctx); mag = 0u - (uint32_t)v; }
            else       { mag = (uint32_t)v; }
            utoa_dec(mag, num);
            put_string(out, ctx, num);
            break;
        }
        case 'u': utoa_dec(va_arg(ap, uint32_t), num); put_string(out, ct
        case 'x': utoa_hex(va_arg(ap, uint32_t), num); put_string(out, ct
        case '%': out('%', ctx); break;
        case '\0': out('%', ctx); return;                    /* string ende
        default:  out('%', ctx); out(*fmt, ctx); break;      /* unknown: as
        }
        fmt++;                                  /* past the conversion letter */
    }
}
```

(The solution file spreads the same code over more lines, with a comment on each.)

Why `0u - (uint32_t) v` and not `-v`? Because the most negative `int`, −2 147 483 648 (`INT_MIN`), has no positive `int` counterpart, so `-v` overflows, which is undefined behavior in C.

Unsigned subtraction is always well defined, and gives 2 147 483 648. After this step `make` must build with no errors.

1.7 The Kernel Print Function (`kprintf`)

With the formatting engine and the UART driver both ready, we can combine them to create a real print function. We name it `kprintf` (where the "k" stands for kernel).

1.7.1 Kernel Mode vs. User Mode Printing

Naming this function `kprintf` highlights a fundamental operating system concept. In a real OS (like Linux), there is a strict separation between how the kernel prints messages and how a user application prints messages:

- **User-Level `printf`:** A user application runs in a restricted user mode. It does not know that a UART even exists, nor does it have the hardware privileges to touch it. When a user program calls `printf`, it triggers a `system call`, asking the OS to print on its behalf. The OS then securely routes that request to the appropriate device driver.
- **Kernel-Level `kprintf`:** The OS kernel itself runs in privileged (supervisor) mode. It inherently knows about the hardware and the I/O devices. Therefore, `kprintf` bypasses system calls entirely and talks directly to the hardware driver. This is not only a design choice: the kernel *cannot* use `printf` for its

own messages, because an `svc` issued from the SVCall handler escalates to a HardFault.

Any operating system that implements hardware protection features both a user-side print function and a kernel-side print function. We are currently inside the operating system layer, so we implement `kprintf` first. (The more complex user-level system call mechanism will be implemented later).

1.7.2 Implementing `kprintf`

In `kernel/kprintf.c`, we write a small wrapper function that connects `vformat` to our polled UART driver.

First, we define the callback function that `vformat` expects. This simply takes a character and hands it to the hardware driver:

```
#include "kernel/kprintf.h"
#include "lib/format.h"
#include "drivers/uart.h"

/* vformat's output: one char to the UART */
static void kprintf_out( char c, void *ctx ){
    (void) ctx;      /* tell the compiler this argument is intentionally u
    uart_putc_polled( c );
}
```

Why `(void) ctx`? The `vformat` engine expects its output function to accept two arguments: the character to print and the context pointer (`ctx`). Our `kprintf_out` function must accept both to match the required signature, even though a simple polled UART driver does not need any context state. Writing `(void) ctx;` is a standard C idiom: it explicitly tells the compiler that the parameter is intentionally ignored, suppressing the 'unused parameter' warning during compilation.

Then, we implement the variadic `kprintf` function itself. It collects its extra arguments into a `va_list` and hands everything over to `vformat`, setting `kprintf_out` as the destination and `0` (NULL) as the context:

```
void kprintf( const char *fmt, ... )
{
    va_list ap;
    va_start( ap, fmt );
    vformat( kprintf_out, 0, fmt, ap );
    va_end( ap );
}
```

How the `va_list` is passed: Notice that `kprintf` itself does not call `va_arg` to read the arguments. It simply initializes the cursor (`ap`) with `va_start`, and passes the whole cursor by value to `vformat`. It is the job of the `switch` statement inside `vformat` to call `va_arg` and advance the cursor as it parses the format string. When `vformat` returns, `kprintf` cleans up the cursor with `va_end`.

Because `kprintf` relies on our polled UART driver, it operates entirely without hardware interrupts. This is a massive architectural advantage: it means `kprintf` can be safely executed anywhere in the kernel, even inside a catastrophic `HardFault` handler where interrupts are strictly disabled.

To test every conversion, we can add a test line in `src/main.c`:

```
kprintf( "c=%c s=%s d=%d u=%u x=%x pct=%%\n",
                    'A', "hi", -42, 4000000000u, 0xBEEFu );
```

This outputs `c=A s=hi d=-42 u=4000000000 x=beef pct=%`. Notice that `%x` prints no `0x` prefix; if you want one, you must write it explicitly in the format string (e.g., `"0x%x"`).

You can follow a call by hand to check the `vformat` logic. For `kprintf("d=%d\n", -42)`:

- The loop passes `d` and `=` to `kprintf_out`.
- It meets `%`, moves to `d`, takes an `int` (-42) with `va_arg`.
- It sends `-`, computes the magnitude 42, turns it into the text `"42"` with `utoa_dec`, and sends `4` and `2`.
- It moves past the `d`, passes `\n`, and stops at the terminating zero.

This results in exactly six calls to `kprintf_out`, pushing six bytes sequentially to the UART.

1.8 Upgrading the Kernel: Fault Handlers That Print

The base's fault handlers are silent infinite loops: if a hardware fault happens, the program simply stops answering. Now that the kernel can print using our robust, polled `kprintf`, the handlers can explicitly state what happened.

1.8.1 The Status Registers (HFSR and CFSR)

Two specific registers of the System Control Block (SCB) tell you why a fault happened:

- **CFSR (Configurable Fault Status Register):** Located at `0xE000ED28`, this register has one specific bit for each possible cause of a `MemManage` fault, a `BusFault`, or a `UsageFault`.
- **HFSR (HardFault Status Register):** Located at `0xE000ED2C`, this indicates exactly why a `HardFault` was escalated. For example, bit 30 (`FORCED`, `0x40000000`) means that a fault was escalated to HardFault.

1.8.2 Implementing the New Handlers

In `kernel/faults.c`, we include the necessary headers and replace the silent loops with explicit print statements before halting:

```
#include "cmsdk.h"
#include "kernel/kprintf.h"

void HardFault_Handler(void) {
    kprintf( "HardFault: HFSR=0x%x\n", SCB->HFSR );
    for(;;){}
}
void MemManage_Handler(void) {
    kprintf( "MemManage: CFSR=0x%x\n", SCB->CFSR );
    for(;;){}
}
void BusFault_Handler(void) {
    kprintf( "BusFault: CFSR=0x%x\n", SCB->CFSR );
    for(;;){}
}
void UsageFault_Handler(void) {
    kprintf( "UsageFault: CFSR=0x%x\n", SCB->CFSR );
    for(;;){}
}
```

1.8.3 Try It: Catching a Divide-By-Zero

To test this new capability, we can intentionally crash the OS. Week 2's `Reset_Handler` set the `DIV_0_TRP` bit in the CCR, meaning an integer division by zero strictly triggers a hardware fault.

Add the following temporary code to `src/main.c`:

```
volatile unsigned zero = 0;      /* read at run time */
kprintf( "%u\n", 10u / zero );   /* should fault     */
```

The `volatile` keyword is crucial here; it forces the compiler to perform the math at runtime rather than computing the result at build time.

Running `make run` yields:

```
UsageFault: CFSR=0x2000000
```

The value `0x2000000` indicates that bit 25 is set, which perfectly corresponds to `SCB_CFSR_DIVBYZERO` in `include/cmsdk.h`. The CPU instantly fired a `UsageFault`, the handler used the polled `kprintf` to push the error string over the UART, and then the system cleanly locked up in the infinite loop.

1.8.4 Practical Note: Project Navigation and Exam Expectations

As we build these low-level layers, the codebase is becoming increasingly complex. Files are spread across different directories (`drivers/`, `kernel/`, `lib/`, `src/`), each serving a distinct architectural role.

From an academic and practical standpoint, it is critical to understand how to navigate this project structure. During an exam (or in a real-world debugging scenario), you will not be asked to write a full operating system from scratch in

90 minutes. Instead, you will typically be given an existing, structured project and asked to implement a specific function, fix a bug, or integrate a new hardware feature. Mastering how to read the existing code, locate the correct files, and understand the flow of execution is just as important as understanding the hardware registers themselves.

1.9 Unit 2: Calling Convention and Interrupts

Up to this point, our entire OS relies on **polling**. The `uart_putc_polled` driver explicitly reads the hardware `STATE` register in an infinite loop until the transmitter has room.

1.9.1 Polling versus Interrupts

While polling is easy to implement, it is unsustainable for a real operating system. The alternative is an **interrupt**: the hardware device sends a physical signal only when it actually needs attention. The CPU pauses instantly, runs a special function called a *handler*, and then resumes exactly where it left off.

	Polling	**Interrupts**
CPU Time	Wasted in an empty waiting loop.	Used only when the event actually occurs.
Latency	Up to one full loop iteration.	The exception entry time ($\sim$12 cycles on Cortex-M3).
Complexity	Trivial; a single loop.	Complex: requires handlers, data protection, and priority management.
CPU State	Constantly locked at 100% usage.	Free to run applications or enter sleep mode.

To understand the real cost of polling: sending 10 bits at 115,200 baud takes about 87 μs. At our board's 25 MHz clock, that equals **2,170 clock cycles**. With polling, the CPU wastes all these cycles just spinning and doing nothing.

However, polling remains vital when interrupts cannot work. This is why our `kprintf` will stay polled: it must guarantee output even when the system crashes inside a `HardFault` (where normal interrupts are strictly blocked).

1.9.2 The AAPCS: How C Calls Functions

To make C and Assembly code work together, both must follow strict rules on how to exchange data. For the ARM architecture, this standard is the **AAPCS** (Procedure Call Standard for the Arm Architecture).

The standard assigns precise roles to the CPU registers:

- `r0-r3`: Carry the first four parameters of a function. The `r0` register also holds the final return value. These are **Caller-Saved** (scratch) registers: the called function can overwrite them freely without asking for permission.

- `r4-r11`: Reserved for the function's local variables. These are **Callee-Saved**: if a function uses them, it is strictly required to back them up (via `push`) and restore them (`pop`) before returning.
- `r12`: A temporary scratch register, strictly **Caller-Saved**.
- `lr` **(Link Register):** Holds the address the CPU must return to when the function ends. Since it is **Caller-Saved**, if a function calls another function, it must save its own `lr` first.
- `sp` **(Stack Pointer):** The stack memory address must always remain a multiple of 8 at every function boundary.

1.9.3 Managing Interrupts from Assembly: The PRIMASK

Before handling specific hardware interrupts, the OS needs a main switch to globally enable or disable the CPU's ability to listen. The Cortex-M3 has a special 1-bit register called **PRIMASK**. When `PRIMASK` is `1`, all configurable interrupts are masked (blocked); when `0`, the CPU is listening.

Standard C cannot touch the CPU's internal registers. We must use specific Assembly instructions:

- `mrs r0, primask`: Reads the value of `PRIMASK` and saves it into `r0`.
- `msr primask, r0`: Writes the content of `r0` into `PRIMASK`.
- `cpsid i` / `cpsie i`: Quick instructions to disable (*disable*) or enable (*enable*) global interrupts.

The Nesting Problem

A naive approach to protecting a critical section of code is to use `cpsid i` at the start and `cpsie i` at the end. However, if a function that disables interrupts calls another function that does the same, the inner function will turn the interrupts back on when it finishes, destroying the protection of the outer function way too early.

The correct solution requires functions to save the previous state before changing it:

```
uint32_t irq_save( void );          /* Blocks interrupts and returns the c
void irq_restore( uint32_t old );   /* Restores PRIMASK to the saved state
```

This way, even with nested calls, interrupts will only be truly re-enabled when the outermost function restores the original `0` value.

Implementing `irq_save` and `irq_restore` in Assembly

We create these two functions in `kernel/irq.s`. Following the AAPCS, `irq_save` must place the old value into the `r0` register before returning to C. Both functions only use `r0` and `lr` (which are Caller-Saved), so we do not need to use `push` or `pop`.

```
    .syntax unified                 /* Modern ARM syntax */
    .cpu cortex-m3                  /* Limit to our chip's instructions */
    .thumb                          /* Use the Thumb instruction set */
```

```
    .section .text,"ax",%progbits

/* uint32_t irq_save(void) */
    .global irq_save            /* Make the name visible to the C linker
    .type irq_save, %function   /* Declare this as executable code */
    .thumb_func
irq_save:
    mrs r0, primask             /* Save the old PRIMASK in r0 (return val
    cpsid i                     /* Set PRIMASK to 1 (mask interrupts) */
    bx lr                       /* Return to C */
    .size irq_save, . - irq_save

/* void irq_restore(uint32_t old) */
    .global irq_restore
    .type irq_restore, %function
    .thumb_func
irq_restore:
    msr primask, r0             /* Restore PRIMASK by reading it from r0
    bx lr                       /* Return to C */
    .size irq_restore, . - irq_restore
```

Testing the API in C

Let's define the prototypes in `kernel/irq.h`:

```
#ifndef IRQ_H
#define IRQ_H
#include <stdint.h>

uint32_t irq_save(void);
void irq_restore(uint32_t state);

#endif
```

And test them inside `src/main.c`:

```
#include "kernel/irq.h"

/* Inside main() */
uint32_t old_state = irq_save();      /* Save current state and block int

/* ---- CRITICAL SECTION START ----
   In here, interrupts are safely blocked.
   ---------------------------------- */

irq_restore(old_state);               /* Restore the previous state */

kprintf("PRIMASK before irq_save was: %u\n", old_state);
```

Execution will print `0`. This confirms the CPU started with interrupts enabled, our assembly safely disabled and cleanly restored them, communicating perfectly

with the C code.

Inspecting the AAPCS with GDB

You can watch live how data travels between C and Assembly using GDB:

```
(gdb) break irq_save
(gdb) continue
(gdb) p/x $lr
(gdb) info registers primask
(gdb) finish
(gdb) info registers r0 primask
```

At the breakpoint, the `lr` (Link Register) contains the return address generated by C, and `PRIMASK` is `0`. The `finish` command runs the assembly function to completion. By checking the registers again, you will see that `r0` now contains the old state (`0x0`) and `PRIMASK` has flipped to `1` (interrupts disabled). Everything moved exactly as dictated by the AAPCS rules.

1.10 ARM Exception Entry and Return Mechanics

While we can wrap many features in simple C functions, handling exceptions requires a deep understanding of what the Cortex-M hardware does autonomously. When a hardware interrupt fires, the CPU abruptly pauses the current execution to jump to an Exception Handler. Before executing the handler's first instruction, the hardware performs three critical operations:

1. It pushes exactly 8 words (the exception frame) onto the current stack.
2. It writes a special signature value, called `EXC_RETURN`, into the Link Register (`lr`).
3. It reads the specific handler's memory address from the vector table and branches to it, automatically elevating to privileged Handler mode.

1.10.1 The Automatic Hardware State Save (Stacking)

The 8 words pushed to the stack form the stacked frame. Because the stack grows downwards, the hardware creates the following memory layout relative to the new Stack Pointer (`sp`):

- `sp + 28`: `xPSR` (Status flags, Thumb bit)
- `sp + 24`: `PC` (Program Counter: where the interrupted code will resume)
- `sp + 20`: `LR` (The interrupted code's Link Register)
- `sp + 16`: `r12`
- `sp + 12`: `r3`
- `sp + 8`: `r2`

- `sp + 4`: `r1`
- `sp + 0`: `r0` ← `sp` when the handler starts

Crucially, these registers (`r0`–`r3`, `r12`, and `lr`) perfectly match the AAPCS *Caller-Saved* set. Because `r4`–`r11` are *Callee-Saved*, any standard C function invoked during the interrupt will safely preserve them before modifying them. Together, the hardware stacking and the AAPCS software rules guarantee that the interrupted application finds all its registers entirely unmodified upon return. This elegant design allows an interrupt handler to be written as a standard C function (e.g., `void SysTick_Handler(void)`) without requiring specialized assembly wrappers or keywords. Additionally, the hardware ensures the stack pointer remains 8-byte aligned upon entry, adhering to the `STKALIGN` bit configured in the CCR during boot.

1.10.2 Exception Return: `EXC_RETURN`

Inside the exception handler, the `lr` does not contain a standard return memory address. Instead, it holds the `EXC_RETURN` value, which begins with `0xFFFFFF...`—an address region deliberately containing no executable code. When the handler concludes and executes a return instruction (like `bx lr` or `pop {..., pc}`), the CPU hardware intercepts this special value and initiates an **exception return**. It pops the 8-word frame from the stack, reverts to the previous execution mode, and seamlessly resumes at the stacked `PC`.

`EXC_RETURN`	**Return To**	**Stack Used**
`0xFFFFFFF9`	Thread mode (normal code, e.g., `main`)	Main stack
`0xFFFFFFF1`	Handler mode (returning to a preempted handler)	Main stack
`0xFFFFFFFD`	Thread mode	Process stack

While the Cortex-M supports a secondary Process Stack Pointer, this kernel exclusively uses the Main stack for now. Therefore, returning to Thread mode will consistently utilize `0xFFFFFFF9`. The compiled C handler ends with an ordinary `bx lr`; the magic lies entirely in the value held within the Link Register.

1.10.3 Tail-Chaining and Late Arrival

To optimize back-to-back exceptions, the hardware utilizes two advanced mechanisms that require no software intervention:

- **Tail-chaining:** If a handler completes while another exception is pending, the CPU skips the redundant process of popping the 8-word frame only to immediately push it again. Instead, it branches directly to the pending handler, reducing latency to about 6 cycles (down from 24) on a Cortex-M3.
- **Late arrival:** If a higher-priority exception arrives while the CPU is actively stacking the frame for a lower-priority exception, the CPU simply uses the partially completed frame, executes the higher-priority handler first, and then tail-chains to the lower-priority one.

In both scenarios, the single stacked frame explicitly describes the original code that was preempted.

1.11 The Heartbeat of the OS: The SysTick Timer

To build an operating system capable of measuring time and preempting tasks, a periodic interrupt is mandatory. The **SysTick** timer is a 24-bit down-counter permanently integrated into every Cortex-M core, mapped to the exact same memory addresses across all chips.

1.11.1 Hardware Preemption

If an OS relies entirely on cooperative software, an infinite loop in a user program would permanently hijack the CPU, deadlocking the system. SysTick prevents this through **hardware preemption**. By periodically firing an interrupt (e.g., every 10 ms), the hardware forcefully pauses the current execution and jumps to the kernel's `SysTick_Handler`. This guarantees the OS regains control at fixed intervals to update the system clock, perform scheduling, or terminate rogue tasks. A hardware interrupt is the only mechanism powerful enough to definitively stop the CPU.

1.11.2 The SysTick Hardware Registers

The timer operates by counting down from a configured `LOAD` value to 0, reloading the `LOAD` value on the subsequent clock cycle. Consequently, exactly one period equals LOAD + 1 cycles.

Register	Address	Function
`CTRL`	`0xE000E010`	Bit 0: **ENABLE** the counter. Bit 1: **TICKINT** (raise the SysTick exception at 0). Bit 2: **CLKSOURCE** (1 = processor clock). Bit 16: **COUNTFLAG** (reads 1 if 0 was reached since last read).
`LOAD`	`0xE000E014`	24-bit reload value (maximum `0xFFFFFF`).
`VAL`	`0xE000E018`	Current count. Writing any value resets it to 0.
`CALIB`	`0xE000E01C`	Read-only calibration data. Bits 23–0 (**TENMS**) hold the reload value for 10 ms on the reference clock.

To initiate the timer, three stores are required:

```
SysTick->LOAD = 249999;                  /* period: 250,000 cycles
                                            (10 ms at 25 MHz) */
SysTick->VAL  = 0;                       /* start from a clean count */
SysTick->CTRL = SysTick_CTRL_ENABLE |
        SysTick_CTRL_TICKINT | SysTick_CTRL_CLKSOURCE; /* = 7 */
```

The SysTick priority is configured using the `SHPR` byte (set to `0xE0` during boot), ensuring it is the least urgent exception in the system. It requires no interrupt-controller setting and no explicit hardware acknowledgment; its pending state clears automatically when the handler starts. While the `CALIB` register allows portable code to achieve a 10 ms tick without knowing the board speed, we know our processor clock is 25 MHz, so we explicitly calculate the `LOAD` value using `CLKSOURCE = 1`.

1.11.3 Building the SysTick Driver

To integrate this hardware into our OS, we write a device driver, structured similarly to the UART driver. The API exposed in `drivers/systick.h` includes initialization and time-tracking functions:

```
void systick_init(void);
uint32_t systick_uptime_ms(void);
```

Choosing the Tick: Reload Value and Rate

The formula for the reload value is **LOAD = (f / rate) - 1**. With our processor frequency (f) at 25 MHz, if we want an interrupt frequency (rate) of 100 Hz (which equals 100 ticks per second, or one tick every 10 ms), the calculation is:

$$\text{LOAD} = \frac{25,000,000}{100} - 1 = 249,999$$

This value (`0x3D08F`) easily fits within the 24-bit limit of the `LOAD` register.

Choosing the tick rate is a fundamental OS design trade-off:

- A faster tick (e.g., 1 ms / 1,000 Hz) measures time more precisely but forces the CPU to process 1,000 exception entries and exits per second, wasting significant CPU time even when the system is idle.
- A slower tick (e.g., 10 ms / 100 Hz) reduces interrupt overhead but lowers timing resolution.

Our kernel utilizes the classic **10 ms (100 Hz)** rate. In QEMU, this specific rate also has the practical benefit of keeping exact time against a physical wall clock, whereas a 1 ms tick runs approximately 26% slower in real-world time due to emulation overhead.

Uptime in Milliseconds and Wraparound Safety

In `drivers/systick.c`, we maintain a global counter tracking the uptime in milliseconds.

```
#include "drivers/systick.h"
#include "cmsdk.h"

#define CPU_CLOCK_HZ 25000000UL      /* processor clock: 25 MHz */
#define TICK_MS 10

static volatile uint32_t uptime_ms; /* changed by the handler */

void systick_init( void )
{
    uptime_ms = 0;
    SysTick->LOAD = CPU_CLOCK_HZ / ( 1000 / TICK_MS ) - 1;
    SysTick->VAL = 0;                    /* any write: counter = 0 */
    SysTick->CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_C
```

```
}

void SysTick_Handler( void )
{
    uptime_ms += TICK_MS;              /* no acknowledge needed */
}

uint32_t systick_uptime_ms( void )
{
    return uptime_ms;                  /* one 32-bit read: never torn */
}
```

The variable `uptime_ms` is strictly declared `volatile` because the `SysTick_Handler` updates it entirely asynchronously. The getter function `systick_uptime_ms` performs a single 32-bit read, which is architecturally guaranteed to be atomic.

Because `uptime_ms` is a `uint32_t`, it wraps from `0xFFFFFFFF` back to 0 after approximately 49.7 days. To safely measure elapsed time without wraparound bugs, the system must *always* use unsigned subtraction:

```
if (now - start >= delay) { /* Safe */ }
```

If `start = 0xFFFFFF00` and the counter wraps such that `now = 0x00000064` (100), the calculation `now - start` safely evaluates to `0x164` (356 ms) thanks to modulo 2^{32} arithmetic.

`wfi`: An Idle Loop That Sleeps

With the hardware timer firing every 10 ms, we can efficiently halt the CPU when it has no immediate work using the `wfi` (Wait For Interrupt) assembly instruction.

```
for ( ;; ) {
    __asm volatile ( "wfi" );  /* sleep until the next interrupt */
    /* ... check whether something has to be done ... */
}
```

The `wfi` instruction instantly puts the processor into a low-power sleep state until any exception is pending. Instead of a polling loop running millions of useless checks per second, a `wfi` loop executes exactly once per interrupt, ensuring the OS remains highly responsive while saving power.

1.12 The Nested Vectored Interrupt Controller (NVIC)

While SysTick is an exception generated internally by the CPU core, interrupts from external peripherals (like the UART or timers on the APB bus) are managed by the **NVIC** (Nested Vectored Interrupt Controller). Acting as a gatekeeper and priority manager, the NVIC sits between these peripherals and the CPU.

On this specific board, the NVIC handles 32 interrupt lines (**IRQ 0 to IRQ 31**). For example, UART0 RX is mapped to IRQ 0, and UART0 TX to IRQ 1.

Following the ARM Cortex-M architecture, an **IRQ n maps to exception number** $16+n$. Its handler address in the vector table is therefore located at offset $4\times(16+n)$. For UART0 TX (IRQ 1), this is exception 17, located at address `0x44`.

The NVIC is configured via memory-mapped registers. Since this board only uses 32 IRQ lines, each of these control registers is a single 32-bit word, where bit n controls IRQ n:

Register	Address	Function (For IRQ n)
`ISER`	`0xE000E100`	**Interrupt Set-Enable.** Writing 1 to bit n enables the IRQ line. Writing 0 has no effect.
`ICER`	`0xE000E180`	**Interrupt Clear-Enable.** Writing 1 to bit n disables the IRQ line. Writing 0 has no effect.
`ISPR`	`0xE000E200`	**Interrupt Set-Pending.** Writing 1 to bit n forces the IRQ into a pending state (software interrupt).
`IPR`	`0xE000E400`	**Interrupt Priority.** Configures the priority level. Only the upper bits are implemented; a lower value means a higher urgency.

To configure the priority and enable UART0 TX, we execute two direct memory stores. Because hardware ignores 0s written to `ISER`, we avoid less efficient read-modify-write (`|=`) operations:

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;
/* Set priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << (UART0TX_IRQn % 32);
/* Enable IRQ 1 (standard CMSIS macro) */
```

1.12.1 The Three Gates of Interrupt Firing

An external device's interrupt will only reach the CPU if it successfully passes through three independent conditions, known as the *three gates*:

1. **The Device Gate:** The peripheral is explicitly configured to generate interrupts (e.g., setting `TXIRQEN` or `RXIRQEN` in the UART `CTRL` register).

2. **The NVIC Gate:** The specific IRQ line is unmasked and enabled in the NVIC (via `ISER`).

3. **The CPU Gate:** Global interrupts are enabled (`PRIMASK = 0`), and the incoming interrupt has a *strictly higher* priority than the currently executing code.

Missing either of the first two gates produces the exact same symptom: the interrupt is completely ignored.

The Golden Rule: Hardware Acknowledgment

NVIC lines are **level-sensitive**. A peripheral continuously asserts its IRQ line as long as its internal interrupt flag (e.g., bit 0 of UART `INTSTATUS`) remains set.

This dictates the Golden Rule of Interrupts: **software must acknowledge and clear the hardware flag at the device level before returning**.

If the handler executes a return (`bx lr`) without clearing this flag, the NVIC still sees the asserted line and immediately fires the interrupt again, permanently trapping the OS in an infinite loop.

Acknowledgment typically involves reading a `DATA` register or writing a 1 to a specific clear-bit. For example, `INTSTATUS` uses write-1-to-clear logic: executing `CMSDK_UART0->INTSTATUS = UART_INT_TX;` clears bit 0 and leaves other flags intact. (Note: SysTick, being a core exception rather than a peripheral, clears its own flag automatically upon execution).

Shared Interrupt Lines (Q&A)

While hardware designers typically map one device to one dedicated IRQ line, complex systems might route multiple devices to a single shared NVIC line to save resources. Because the CPU only sees a single asserted wire, the software handler must manually poll the status registers of *every* device sharing that line to determine which one triggered the interrupt, service it, and clear its specific flag.

1.13 Expanding the Vector Table for External Devices

The base's `boot/startup.s` allocates 16 core slots plus 32 hardware IRQs, totaling 48 words (192 bytes, or `0xC0`).

Devices are assigned specific names (e.g., `UART0TX_Handler` is IRQ 1 at word 17; `TIMER0_Handler` is IRQ 8 at word 24). Unused slots point to a `Default_Handler`. Each handler name uses `.weak` and `.thumb_set` directives: this allows a C function defined with the identical name to automatically override the weak default handler.

You can verify this in GDB: examining `x/4wx 0x40` (slots 16–19) reveals identical words pointing to `Default_Handler` (plus 1 for the Thumb state).

1.14 Concurrency and Shared Data

When we enable hardware interrupts, we introduce **concurrency**. An interrupt can literally pause the `main` program at any given nanosecond—even right in the middle of a simple C instruction. This forces us to carefully manage how the main program and the interrupt handlers share memory.

1.14.1 The Problem: Race Conditions

Imagine the `main` loop and an interrupt handler both want to update a shared variable by doing `count++`. While it looks like a single step in C, the CPU actually performs three separate assembly instructions:

```
ldr  r1, [r0]      /* 1. READ: load current value (e.g., 5) into r1 */
                   /* ---- WHAT IF AN INTERRUPT FIRES RIGHT HERE? ---- */
adds r1, r1, #1    /* 2. ADD: increment r1 to 6 */
str  r1, [r0]      /* 3. WRITE: store 6 back into memory */
```

If an interrupt fires exactly after the read step, the handler might run, increment `count` to 6, and return. But the `main` program, completely unaware, resumes its sequence holding the old, stale value. It will calculate $5 + 1 = 6$ and write it back, effectively overwriting and erasing the handler's work. This is a **race condition**, resulting in a **lost update**.

1.14.2 The Solution: Critical Sections

To protect shared data, we must create a **critical section**: a small block of code where the CPU temporarily ignores interrupts. We do this using our `irq_save()` and `irq_restore()` functions.

Three golden rules apply here:

1. **Asymmetry:** An interrupt handler cannot be interrupted by `main`. Therefore, only the `main` program needs to protect itself by masking interrupts.

2. **Brevity:** Keep the critical section as short as possible. Do not put delays or loops inside it. If an interrupt arrives during this time, the hardware will safely keep it "pending" until you restore the mask.

3. **Proper Nesting:** Always end with `irq_restore(s)` instead of a raw "enable" instruction. This safely restores the *previous* state, in case the function was called from another already-protected section.

1.14.3 The Ring Buffer (Circular Queue)

To send a long text message asynchronously, `main` needs a place to "drop off" the characters so the handler can pick them up later. The best structure for this is a **ring buffer**: an array that wraps around itself like a circle.

We define this in the internal state of our UART driver (**File:** `drivers/uart.c`):

```
/* File: drivers/uart.c */
#define TX_BUF_SIZE 64

static volatile char tx_buf[TX_BUF_SIZE];
static volatile unsigned tx_head = 0; /* Written ONLY by main (Producer)
static volatile unsigned tx_tail = 0; /* Written ONLY by handler (Consume
static volatile int tx_busy = 0;
```

The math relies on keeping `TX_BUF_SIZE` a power of two (like 64). This turns the slow modulo division (`%`) into a lightning-fast bitwise AND (`&`):

- **Next index:** `(i + 1) & (TX_BUF_SIZE - 1)`
- **Buffer Empty:** `head == tail`
- **Buffer Full:** `((head + 1) & (TX_BUF_SIZE - 1)) == tail`

Notice that out of 64 slots, we can only safely use **63**. We purposely leave one slot permanently empty so the "full" state never mathematically looks like the "empty" state.

1.14.4 Interrupt-Driven Transmission

Now we rewrite our UART driver to use the ring buffer. The workload is split into two parts: a **Producer** (the main program) and a **Consumer** (the hardware interrupt).

The Producer: `uart_write`

This function is called by the application to send text. Its job is to safely shove as many characters as possible into the ring buffer.

It also has one vital responsibility called the **Priming Rule**. If the UART hardware is currently idle, it isn't generating any interrupts. We must "prime the pump" by manually turning the interrupt on and sending the very first character.

```
/* File: drivers/uart.c */
unsigned uart_write(const char *buf, unsigned len)
{
    unsigned accepted = 0;

    /* CRITICAL SECTION START: Mask interrupts */
    uint32_t saved = irq_save();

    /* 1. Fill the buffer as much as possible */
    while (accepted < len && !tx_full()) {
        tx_buf[tx_head] = buf[accepted];
        tx_head = (tx_head + 1) & (TX_BUF_SIZE - 1);
        accepted++;
    }

    /* 2. The Priming Rule: Kickstart the hardware if it's asleep */
    if (!tx_busy && !tx_empty()) {
        /* FIRST: Enable the hardware interrupt */
        CMSDK_UART0->CTRL |= UART_CTRL_TXIRQEN;

        /* Wait just in case a previous byte is still physically leaving
        while (CMSDK_UART0->STATE & UART_STATE_TXBF) {}

        /* THEN: Write the first byte to trigger the cycle */
        CMSDK_UART0->DATA = (uint32_t)(unsigned char)tx_pop();
        tx_busy = 1;
    }

    /* CRITICAL SECTION END: Restore interrupts */
    irq_restore(saved);

    return accepted; /* Tell main how many bytes actually fit */
}
```

The Consumer: `UART0TX_Handler`

The handler wakes up completely automatically every time a character successfully leaves the hardware. Since it is an interrupt, it inherently blocks `main` from running, so it doesn't need `irq_save()`.

Its logic is simple: Acknowledge the hardware, fetch the next byte from the tail of the buffer, and send it. If there's nothing left, turn the lights off and go to sleep.

```
/* File: drivers/uart.c (or kernel/handlers.c depending on your architect
void UART0TX_Handler(void)
{
    /* 1. Acknowledge the interrupt at the hardware level (Write 1 to Cle
    CMSDK_UART0->INTSTATUS = UART_INT_TX;

    /* 2. Is there more data waiting in the ring buffer? */
    if (!tx_empty()) {
        /* Yes: Send the next byte */
        CMSDK_UART0->DATA = (uint32_t)(unsigned char)tx_pop();
    } else {
        /* No: Turn off the TX interrupt and mark as idle */
        CMSDK_UART0->CTRL &= ~UART_CTRL_TXIRQEN;
        tx_busy = 0;
    }
}
```

Initialization: `uart_init`

During system boot, we must set up the buffer state and configure the NVIC priorities. We assign this interrupt a priority of `0xC0` (lower urgency than System Calls, but higher than the SysTick timer).

```
/* File: drivers/uart.c */
void uart_init(void)
{
    /* ... (standard baud rate setup) ... */

    /* Reset the ring buffer state */
    tx_head = 0;
    tx_tail = 0;
    tx_busy = 0;

    /* Set Priority to 0xC0 */
    NVIC->IPR[UART0TX_IRQn] = 0xC0;

    /* Enable IRQ 1 (UART0 TX) in the NVIC Gate */
    NVIC->ISER[UART0TX_IRQn / 32] = 1UL << (UART0TX_IRQn % 32);
}
```

1.14.5 Testing the Driver

To prove this works, let's try to send 160 bytes of text. Since our buffer can only hold 63 bytes at a time, `uart_write` will inevitably fill it up and return early.

Instead of polling in a busy loop and wasting CPU cycles, we use the `wfi` **(Wait For Interrupt)** instruction to put the processor to sleep. Every time the handler finishes sending a byte, it wakes the CPU up, allowing us to check if there is room for the rest of the text.

```
/* File: src/main.c */
#include "drivers/uart.h"

/* 4 lines of 40 bytes = 160 bytes total (too big for the 64-byte buffer)
static const char msg[] =
    "line 1: the quick brown fox jumps over.\n"
    "line 2: the quick brown fox jumps over.\n"
    "line 3: the quick brown fox jumps over.\n"
    "line 4: the quick brown fox jumps over.\n";

int main(void)
{
    uart_init();

    const char *p = msg;
    unsigned left = sizeof(msg) - 1; /* Ignore the null terminator */

    while (left > 0) {
        /* Try to shove as much as possible into the buffer */
        unsigned n = uart_write(p, left);

        p += n;       /* Move the pointer forward by 'n' bytes */
        left -= n;    /* Decrease the remaining bytes count */

        /* If the buffer was completely full (0 bytes accepted)... */
        if (n == 0) {
            /* Go to sleep! The UART handler will wake us up
               when it frees up some space. */
            __asm volatile ("wfi");
        }
    }

    while(1) {
        __asm volatile ("wfi"); /* Idle forever once finished */
    }
}
```

What happens under the hood: `uart_write` will run 3 times, successfully copying 63, then 63, then 34 bytes. Meanwhile, the `UART0TX_Handler` will automatically trigger exactly 160 times in the background, smoothly transferring the entire string byte-by-byte while the `main` program literally sleeps through most of the process.

End of Unit 2

At this stage, `drivers/uart.c`, `drivers/systick.c`, `kernel/irq.s`, and `boot/startup.s` are functionally complete.

The `src/main.c` file currently initializes the UART, prints the kernel banner, starts the SysTick timer, sends the 160-byte test message through the interrupt-driven ring buffer, and then runs the temporary privileged loop that prints the uptime every second.

Currently, everything still runs privileged, directly inside `main`. The final architectural phase (Unit 3) will move this application workload to an unprivileged environment and establish a real, secure kernel boundary between the user application and the hardware.

1.15 Unit 3: System Calls and the Kernel Boundary

At the end of Unit 2 everything still ran privileged inside `main`. In this unit we build a real boundary: the application runs **unprivileged** and can reach the hardware only by *asking* the kernel, through **system calls** built on top of the `svc` exception.

1.15.1 Recap: Privilege and `svc`

From Week 2, we established four architectural facts:

1. `main` starts **privileged**. Setting `CONTROL.nPRIV = 1` drops the CPU into **unprivileged** Thread mode. The transition is one-way: changing `CONTROL` is itself a privileged operation, so unprivileged code can never clear the bit again.

2. Unprivileged code that touches the System Control Space (SysTick, NVIC, SCB) triggers a **BusFault**.

3. The `svc #n` instruction is always permitted and raises the **SVCall** exception.

4. Every exception handler runs in Handler mode, which is **privileged**. When the handler returns, the CPU resumes right after the `svc` instruction and drops back to unprivileged mode.

Put together, `svc` is the only controlled doorway from unprivileged to privileged code: the application can knock, but the kernel's handler decides what happens.

In Week 2, `SVC_Handler` performed one fixed job (printing a single message). In this unit it becomes a **system-call interface**: it multiplexes several services, receives arguments, and returns results to user space.

1.15.2 The Rule From Now On: The App Asks, The Kernel Does

In our architecture, the application never calls a device driver directly and never touches a hardware register. **Every I/O operation crosses the kernel boundary via a system call**, just as in Linux a user-space `printf` ultimately invokes the `write` system call.

The diagram follows one request from top to bottom. Each layer talks only to the layer right below it, and the double line marks the kernel boundary: above it the code runs unprivileged in Thread mode, below it privileged in Handler mode.

```
 app_main()          printf("uptime: %u ms\n", now)        src/app.c          \
     |                                                                          | un
 user library        printf -> write_all -> sys_write()    lib/printf.c       | (Th
     |                                                                          |
 user stub           svc #1 ; bx lr                        lib/syscalls.s     /
=====+============== SVCall exception: the kernel boundary ======================
 entry               SVC_Handler: mrs r0, msp ; b ...      kernel/svc_entry.s  \
     |                                                                         | p
 dispatcher          svc_dispatch(frame): table[n](...)    kernel/syscall.c    | (H
     |                                                                         |
 driver              uart_write() + UART0TX_Handler        drivers/uart.c      /
     |
 hardware            UART0 registers at 0x40004000
```

Logical vs. Physical Protection

Our Cortex-M3 setup does not yet use a Memory Protection Unit (MPU). The hardware therefore blocks unprivileged access only to the System Control Space (*physical* protection); the UART registers at `0x40004000` remain reachable from unprivileged code, so keeping the app away from them is for now only a *logical* rule of our design.

Even so, the kernel boundary is a hard technical requirement, not a mere convention. The UART driver uses `irq_save()` (which executes `cpsid i`) to protect its ring buffer. In unprivileged mode `cpsid` is silently ignored: the driver would believe it is protected while interrupts stay enabled. The driver is therefore unsafe unless it runs in privileged kernel mode.

1.15.3 The System-Call ABI and Services

While an API (Application Programming Interface) defines rules at the C source code level (like function names and types), an **ABI (Application Binary Interface)** is a strict low-level contract between the compiler and the hardware. When the C code is compiled into machine instructions, variable names disappear, leaving only memory addresses and CPU registers.

For our system calls, the ABI dictates exactly how the User Space and the Kernel must pass messages to each other:

- **Service Number:** Embedded directly inside the `svc #n` assembly instruction.
- **Arguments:** The User Space must place them in registers `r0`, `r1`, and `r2` (following the AAPCS) before calling the kernel.
- **Result:** The Kernel must leave the final answer in `r0`, exactly where the C caller expects to find it.

In practice, a system call looks exactly like an ordinary C function to the programmer, except under the hood it triggers an `svc` (Supervisor Call) instead of a standard branch (`bl`).

To avoid using magic numbers in our C code, we define symbolic constants (e.g., `SYS_WRITE`, `SYS_READ`) in a shared header (`lib/syscalls.h`) included by both user space and the kernel. The available services are:

Number	Service	Arguments	Result
1	`sys_write`	`buf`, `len`	Bytes **accepted** (0 to `len`). Routes to `uart_write`.
2	`sys_read`	None	One received byte, or `-1` if none. Routes to `uart_getc_polled`.
3	`sys_uptime`	None	Milliseconds since boot. Routes to `systick_uptime_ms`.
$0, \geq 4$	Invalid	None	Unknown service (e.g., 0 or 99): returns `-1`.

No service waits: `sys_write` takes only what currently fits in the TX ring buffer and returns the count immediately. The unprivileged caller is responsible for waiting (the reason is explained in "Why the Kernel Never Waits" below).

1.15.4 The Exception Frame and the Journey of a Syscall

In software engineering, a **stub** is a small "dummy" function. From the perspective of the user application, asking to print a string is as simple as calling `sys_write(buf, len)`. However, this C function is actually an assembly stub that does not perform any real work. Its only job is to pause the application and securely request the kernel's attention.

To understand how data safely crosses the boundary between User Space and the Kernel, we must track the exact sequence of events:

1. **C prepares the data (User Space):** Following the ABI, the C compiler places the arguments `buf` and `len` into physical registers `r0` and `r1`.

2. **The Stub traps the CPU (Assembly):** The stub executes `svc #1`. The Cortex-M hardware instantly takes over, suspends the application, and automatically saves 8 core registers (including `r0` and `r1`) into the stack memory. This memory block is called the **Exception Frame**.

3. **The Kernel executes (C):** The kernel views this saved memory block as a standard C array (`uint32_t *frame`). The magic of this array is its layout: `frame[0]` is exactly where the hardware saved the old `r0`, and `frame[1]` is `r1`. The kernel simply reads `frame[0]` and `frame[1]` to retrieve the user's arguments, performs the UART write, and writes its final result back into `frame[0]`, overwriting the old data.

4. **The Hardware Restores the Context:** The kernel finishes its job. The hardware retrieves the Exception Frame from memory and shoots the values back into the physical CPU registers. Because the kernel wrote the result into `frame[0]`, the physical register `r0` is now populated with the kernel's answer.

5. **The Stub returns (User Space):** The C program wakes up. It checks `r0` (as standard C functions do) and seamlessly finds the return value, completely unaware that a complex kernel transition just occurred.

Hardcoded Syscall IDs (The Assembly Stubs)

Because this pure assembly file (`lib/syscalls.s`) cannot easily read the C macros defined in `lib/syscalls.h`, the service numbers following the `svc` instruction are encoded as hardcoded literals.

```
    .syntax unified
    .cpu cortex-m3
    .thumb
    .section .text,"ax",%progbits

/* int sys_write(const char *buf, unsigned len)
   C places 'buf' in r0, 'len' in r1 */
    .global sys_write
    .type sys_write, %function
    .thumb_func
sys_write:
    svc #1                      /* Trap into kernel requesting service #1
    bx lr                       /* Return to C (kernel placed result in r
    .size sys_write, . - sys_write

/* int sys_read(void) */
    .global sys_read
    .type sys_read, %function
    .thumb_func
sys_read:
    svc #2                      /* Trap into kernel requesting service #2
    bx lr                       /* Return to C */
    .size sys_read, . - sys_read

/* uint32_t sys_uptime(void) */
    .global sys_uptime
    .type sys_uptime, %function
    .thumb_func
sys_uptime:
    svc #3                      /* Trap into kernel requesting service #3
    bx lr                       /* Return to C */
    .size sys_uptime, . - sys_uptime
```

1.15.5 The Kernel Entry Point: Why It Must Be Assembly

When the `svc` instruction fires, the CPU jumps to the `SVC_Handler`. Its only responsibility is to find the exact memory address of the Exception Frame and pass it to the C kernel dispatcher (`svc_dispatch`). That address is simply the current Stack Pointer.

This is exactly why `SVC_Handler` **must** be written in raw assembly (`kernel/svc_entry.s`). C compilers are highly intrusive: when you write a C function, the compiler automatically inserts hidden "prologue" code (such as `push {r7, lr}`) to allocate local variables. This prologue shifts the Stack Pointer downward without our knowledge. If `SVC_Handler` were written in C, we would instantly lose track of where the hardware actually saved the Exception Frame.

By writing it in assembly, we achieve millimeter-level precision. The very first instruction executes exactly as written:

```
    .global SVC_Handler
    .type SVC_Handler, %function
    .thumb_func
SVC_Handler:
    mrs r0, msp                 /* r0 = MSP: save the exact address of th
    b svc_dispatch              /* Jump to C code, passing the frame addr
```

The `mrs r0, msp` instruction securely captures the uncorrupted Stack Pointer and passes it as the first argument (`r0`) to `svc_dispatch`.

Notice we use a plain branch (`b`) rather than a branch-with-link (`bl`). A `bl` instruction would overwrite the Link Register (`lr`), destroying the vital `EXC_RETURN` value (`0xFFFFFFF9`) needed to return to User Space. With a tail branch, `lr` is preserved, allowing the hardware to perform the exception return seamlessly when the C kernel completes its task.

1.15.6 Extracting the Service Number and Arguments

The service number travels inside the `svc` instruction itself, so the kernel has to read it back from the code. In Thumb mode, `svc #n` is a **16-bit** instruction: the high byte is the opcode `0xDF` and the low byte is the immediate `n` (so `svc #1` is `0xDF01`). Because the Cortex-M3 is **little-endian**, the low byte `n` is stored at the lower address, followed by `0xDF`:

```
address:    0x8e0    0x8e1    0x8e2    0x8e3
byte:       0x01     0xdf     0x70     0x47
            +- svc #1 --+     +- bx lr ---+
```

On exception entry, the hardware pushes the 8-word frame: `frame[0..7]` = `r0-r3`, `r12`, `lr`, `pc`, `xPSR`. The stacked `PC` (`frame[6]`) points to the instruction immediately *after* `svc` (here `bx lr`). Working backwards from it, the opcode `0xDF` sits at `PC - 1` and the service number `n` sits at `PC - 2`. The user arguments (`r0-r2`) are read from `frame[0]`, `frame[1]`, and `frame[2]`.

1.15.7 The Dispatcher and Validating the Result Return

To route calls without a large `switch`, `kernel/syscall.c` uses a function pointer table:

```
typedef uint32_t (*syscall_fn)(uint32_t a0, uint32_t a1, uint32_t a2)

static const syscall_fn table[] = {
    [SYS_WRITE]   = sys_write_impl,
```

```
    [SYS_READ]    = sys_read_impl,
    [SYS_UPTIME]  = sys_uptime_impl,
};
#define TABLE_SIZE ( sizeof( table ) / sizeof( table[0] ) )

void svc_dispatch( uint32_t *frame )
{
    uint8_t n = ( ( uint8_t * ) frame[6] )[-2];

    if ( n >= TABLE_SIZE || table[n] == 0 ) {
        frame[0] = ( uint32_t ) -1;              /* unknown service */
        return;
    }
    /* run it; the result goes into the stacked r0 = the caller's r0 */
    frame[0] = table[n]( frame[0], frame[1], frame[2] );
}
```

How it works:

- The table uses designated initializers: each service number is the *index* of its implementation. Index 0 is never filled, so it stays empty (`NULL`).
- `n` is read from the byte at stacked `PC - 2`, as seen above.
- The check `n >= TABLE_SIZE || table[n] == 0` rejects out-of-range numbers (e.g., 99) and the empty slot 0: the caller receives `-1`.
- Otherwise the implementation is called with the three stacked arguments, and its return value is stored in `frame[0]`.

Why Write to `frame[0]`? The kernel cannot return the result simply by leaving it in the physical register `r0`. On exception return, the hardware pops the 8-word frame from the stack back into the CPU registers, overwriting physical `r0` with `frame[0]`. Writing the return value into `frame[0]` ensures the hardware restores it into the caller's `r0`.

Try It: Your First System Call

In `src/main.c`, include `"lib/syscalls.h"` and replace `systick_uptime_ms()` with `sys_uptime()`. The output is unchanged, but it now follows the new path:
`sys_uptime` → `svc #3` → `SVC_Handler` → `svc_dispatch` → `sys_uptime_impl` → `systick_uptime_ms`.
(Although `main` is still privileged here, `svc` works from privileged mode too.)

Finding the Service Number in GDB

You can inspect the hardware frame and decode n in GDB:

```
(gdb) break *SVC_Handler                # very first instruction of the e
(gdb) continue
(gdb) x/8wx $sp                         # the frame: r0 r1 r2 r3 / r12 lr
```

```
(gdb) p/x $lr
(gdb) x/2bx ((unsigned *)$sp)[6] - 2    # the 2 bytes before the stacked
```

Here `frame[6]` points right after `svc` (e.g., `0x7e6`), and the two preceding bytes are `0x03` and `0xDF` (`svc #3` in little-endian, so $n = 3$, `SYS_UPTIME`). Moreover, `lr` holds `0xFFFFFFF9` (`EXC_RETURN`), preserved by `b svc_dispatch`. Running `stepi` twice enters `svc_dispatch`, and `finish` returns directly to `bx lr` in `sys_uptime`.

1.15.8 The Kernel Rules: Where SVC Works

The `svc` instruction raises `SVCall` synchronously, but like any exception it can only preempt code that is *less* urgent than itself (remember: a lower priority value means higher urgency). SVCall has priority `0x80`:

Where `svc` runs	**Current priority**	**Result**
`app_main`, `main` (Thread mode)	none (lowest)	taken
`SysTick_Handler` / `UART0TX_Handler`	`0xE0` / `0xC0`	taken: SVCall is more urgent, so it nests
`SVC_Handler` and anything it calls	`0x80`	**HardFault**
a fault handler	`0x00` or more urgent	**HardFault**
between `irq_save` and `irq_restore` (`PRIMASK = 1`)	masked	**HardFault**

Executing `svc` from a handler of equal or higher urgency (or while `PRIMASK = 1`) cannot be serviced, and it escalates to a **HardFault**. This is why the kernel retains its own polled `kprintf`, independent of system calls.

1.15.9 Why the Kernel Never Waits

The kernel must never wait inside an exception handler. Suppose `sys_write_impl` looped inside `SVCall` (priority `0x80`) waiting for the UART TX buffer to empty: the UART TX interrupt (priority `0xC0`) is less urgent, so it could never preempt the loop, and the buffer would never empty. The system would deadlock. SysTick (`0xE0`) would be blocked as well, stopping the system clock.

Instead, the kernel accepts only what fits in the buffer and returns immediately, leaving the waiting (`wfi`) to unprivileged Thread mode, where every interrupt can run. Two experiments at the end of this unit show both problems live.

1.15.10 User Space Library: Implementing `printf`

With the system calls operational, we move to user space and implement `printf` as a user-level library in `lib/printf.c`. The kernel keeps its own polled `kprintf`; user applications use this `printf`, built on system calls.

Unlike `kprintf` (which calls the polled hardware driver directly), the user-level `printf` works under unprivileged constraints, in two steps:

1. It uses the shared `vformat` engine to format the string into a local 64-byte buffer allocated on the unprivileged task's stack.

2. It flushes this buffer to the hardware by invoking the `sys_write` system call.

	`kprintf`	`printf`
Runs in	Kernel (privileged)	User code (unprivileged)
Output path	Polled UART driver, directly	`vformat` → `sys_write` → `svc` → ring buffer
When output is blocked	Busy-waits (polling)	Sleeps with `wfi`
Inside handlers / HardFault	Works: needs no `svc`	**HardFault**: the `svc` cannot be taken

Handling Partial Writes and Waiting (`write_all`)

Because `sys_write` pushes data into the kernel's 64-byte interrupt-driven ring buffer, it only accepts what currently fits and returns the accepted count immediately, without waiting.

The user library handles partial writes with `write_all`, which loops until all bytes are sent. If `sys_write` returns 0, the ring buffer is full: instead of busy-waiting, the application executes `wfi` (Unit 2). The CPU sleeps until an interrupt fires (such as UART TX freeing a buffer slot), then resumes the loop:

```
void write_all(const char *buf, unsigned len) {
    while ( len > 0 ) {
        int n = sys_write(buf, len); /* The kernel takes what fits */
        buf += n;                    /* Skip the accepted bytes */
        len -= n;

        if (n == 0) {
            __asm volatile ("wfi");  /* Buffer full: sleep until an inter
        }
    }
}
```

1.15.11 Implementing `sleep_ms` in User Space

Using `wfi` and unsigned subtraction (Unit 2), we can implement `sleep_ms` without imprecise delay loops, by checking `sys_uptime()` (`SYS_UPTIME`) between sleep states:

```
void sleep_ms(uint32_t ms) {
    uint32_t start = sys_uptime();
    while (sys_uptime() - start < ms) {
        __asm volatile ("wfi"); /* Sleep until the next interrupt */
    }
}
```

The condition `sys_uptime() - start < ms` uses unsigned 32-bit arithmetic, which handles the 32-bit wraparound (after $\approx$ 49.7 days) without error. Since SysTick fires every 10 ms, each `wfi` wakes the task at least that often to re-check the clock. Because the waiting occurs in unprivileged Thread mode, the kernel is never blocked: every interrupt can preempt the sleeping task, allowing the OS to handle background UART transmission and timekeeping.

1.15.12 The Final System Boot Flow

`src/main.c` acts as the **kernel** initializer. It runs privileged once at boot and performs five steps:

1. `uart_init()`: Initializes the console first so subsequent code can print.

2. `kprintf`: Prints the boot banner via polling.

3. `systick_init()`: Starts the 10 ms tick used by `sys_uptime` and `sleep_ms`.

4. **Privilege Drop**: Sets `CONTROL.nPRIV = 1` followed by an `isb` (Instruction Synchronization Barrier). The `isb` flushes any instructions already prefetched under the old privileged state, ensuring all subsequent instructions run unprivileged.

5. `app_main()`: Calls the user application, which never returns.

```
#include "drivers/uart.h"
#include "drivers/systick.h"
#include "kernel/kprintf.h"
#include "src/app.h"

int main( void )
{
    uart_init();                                    /* 1. console first */
    kprintf( "scratch-os: kernel started\n" );      /* 2. polled print */
    systick_init();                                 /* 3. 10 ms tick */
    kprintf( "dropping to unprivileged mode\n" );

    __asm volatile (                /* 4. CONTROL = CONTROL | 1; (nPRIV = 1)
        "mrs r0, control     \n" /* r0 = CONTROL */
        "orr r0, r0, #1      \n" /* r0 |= 1: set nPRIV */
        "msr control, r0     \n" /* CONTROL = r0: now unprivileged */
        "isb                 \n" /* refetch under the new rules */
        : : : "r0", "memory" );

    app_main();                                     /* 5. never returns */
    for ( ;; ) { }                                  /* not reached */
}
```

1.15.13 The User Application: `app_main`

`src/app.c` contains the unprivileged application, which includes only `lib/syscalls.h` and `lib/printf.h`.

Every 1000 ms, it prints the uptime. It compares against the system clock (`now - last >= 1000`) rather than counting loop iterations, which would drift because of the time spent inside `printf`. On each iteration, it also checks `sys_read()` for incoming keys, echoes them if present, and sleeps via `sleep_ms(10)`:

```
#include <stdint.h>
#include "src/app.h"
#include "lib/syscalls.h"
#include "lib/printf.h"

void app_main( void )
{
    printf( "app: hello from unprivileged code\n" );
    uint32_t last = 0;          /* when the uptime was last printed */

    for ( ;; )
    {
        uint32_t now = sys_uptime();
        if ( now - last >= 1000 )           /* one second passed? */
        {
            printf( "uptime: %u ms\n", now );
            last += 1000;
        }

        int c = sys_read();                 /* never waits: -1 = no key
        if ( c != -1 )
        {
            printf( "key: %c\n", c );
        }

        sleep_ms( 10 );
    }
}
```

While `app_main` runs unprivileged, hardware interrupts (SysTick, UART TX) and system calls (SVCall) still execute privileged: the Cortex-M hardware raises privilege on exception entry and restores unprivileged mode on return.

1.15.14 A Debug Print Inside the Kernel

This experiment answers a question: why does the kernel keep its own `kprintf`, now that there is a `printf`? Suppose you want to trace every uptime request. In `kernel/syscall.c`, add `#include "lib/printf.h"` below the other includes, and in `sys_uptime_impl` add one line before the `return`:

```
    printf("uptime called\n");          /* NEW: debug print */
```

You might expect `uptime called` about a hundred times per second. `make run` prints instead:

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
HardFault: HFSR=0x40000000
```

Follow the call: `printf` → `write_all` → `sys_write` → `svc #1`, executed **inside** the SVCall handler, whose priority is `0x80`. An exception can only interrupt code that is **less urgent** than itself, and SVCall is not more urgent than SVCall. Unlike an interrupt, which can simply stay pending, an `svc` cannot be postponed: it must be taken now or never. So the CPU **escalates** it to a HardFault.

`HFSR` bit 30, **FORCED** (`0x40000000`, `SCB_HFSR_FORCED` in `include/cmsdk.h`), means "a fault was escalated to HardFault". Notice that the report itself prints fine: `HardFault_Handler` uses `kprintf`, which needs no `svc`. Undo both edits (the `#include` and the `printf` line).

1.15.15 Making the Kernel Wait

The second experiment answers another question: why does `sys_write` return a count instead of simply waiting until everything is sent? Make two temporary edits. In `kernel/syscall.c`, in `sys_write_impl`, replace `return uart_write(buf, len);` with a loop that waits until **everything** is accepted:

```
unsigned done = 0;
while (done < len)
    done += uart_write(buf + done, len - done);
return done;
```

And in `src/app.c`, in `app_main`, right after the hello `printf`, add a direct write of 130 bytes, more than the 64-byte ring buffer (the compiler joins the four adjacent strings into one):

```
sys_write("01234567890123456789012345678901234567890123456789"
          "01234567890123456789012345678901234567890123456789"
          "01234567890123456789012345678901234567890123456789"
          "012345678\n", 130);
```

Do the 130 characters appear? Do the `uptime:` lines? `make run` prints:

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
0
```

One `0`, then nothing, forever. In GDB (`make debug`, `continue`, then `Ctrl-C` to interrupt), `p $xpsr & 0x1ff` gives `11`: the CPU is inside the SVCall handler. `bt` shows `uart_write` ← `sys_write_impl` ← `svc_dispatch` ← `<signal handler called>` ← `sys_write` ← `app_main` (the innermost frame may be `irq_save` or `irq_restore` instead, depending on where you interrupt it).

Here is what happened:

1. The first `uart_write` filled the ring buffer and **primed** the UART with the first byte: that is the `0` you see.

2. From then on the buffer stays full. Only `UART0TX_Handler`, at priority `0xC0`, can empty it, but the CPU is inside the SVCall handler, at `0x80`, which is more urgent.

3. The TX interrupt stays **pending** forever, and the loop waits forever for room that never comes.

4. SysTick, at `0xE0`, is blocked too: the clock stops.

The fix is the design you already have. The kernel **never waits**: `sys_write` returns the accepted count, and `write_all` does the waiting with `wfi` in **Thread mode**, where the TX interrupt can run. Undo both edits.

1.15.16 Privilege and Layers in GDB

We can verify the privilege separation in GDB by setting breakpoints in `main` and `app_main` and inspecting `$control`:

```
(gdb) break main
(gdb) break app_main
(gdb) continue          -> Breakpoint 1, main ()
(gdb) p/x $control      -> 0x0          (privileged)
(gdb) continue          -> Breakpoint 2, app_main ()
(gdb) p/x $control      -> 0x1          (nPRIV set: unprivileged)
```

Next, place a breakpoint inside `sys_write_impl`:

```
(gdb) break sys_write_impl
(gdb) continue
(gdb) p/x $control              # CONTROL: is nPRIV still set?
(gdb) p/x $xpsr & 0x1ff         # low 9 bits of xPSR = exception number
(gdb) bt                        # backtrace: every layer of the call path
```

Inside `sys_write_impl`, `$control` is still `0x1` (`nPRIV` remains set); the CPU has privileged access because it is running in Handler mode, not because `CONTROL` changed. The low 9 bits of `xPSR` hold the exception number, where `0xb` (11) identifies SVCall.

The backtrace (`bt`) shows every layer of the call path:

```
#0  sys_write_impl (a0=..., a1=34, a2=...)
#1  svc_dispatch (frame=...)
#2  <signal handler called>
#3  sys_write ()
#4  write_all (buf=... "app: hello from unprivileged code\n", len=34)
#5  printf (fmt=...)
#6  app_main ()
#7  main ()
```

The `<signal handler called>` frame is GDB's marker for an **exception entry**: frames above it run in privileged Handler mode, while frames below it belong to the unprivileged user application.

1.16 Where to Find the Code

`week-03/` holds two sibling folders, `base/` and `solution/`, each with its own `Makefile` and building independently. All paths in these notes are relative to `week-03/solution/`. The base's `README.md` lists, folder by folder, what is given and what you write across the three units.

The starting point is `week-03/base/`: the full `Makefile` (`run`, `debug`, `run-trace`), the linker script, the complete `include/cmsdk.h`, `boot/startup.s` with CPU setup and the 16 core slots, silent fault handlers in `kernel/faults.c`, every API header, the number helpers in `lib/format.c`, a `src/main.c` that prints nothing, and marker files for everything else.

The finished result is in `week-03/solution/`, organized by OS layer:

- `boot/startup.s`: the 48-entry vector table `isr_vector`, the weak defaults, and `Reset_Handler`;
- `include/cmsdk.h`: the hardware register map (Unit 1);
- `drivers/uart.c` (**driver**): `uart_init`, `uart_putc_polled`, `uart_getc_polled`, the TX ring buffer, `uart_write`, and `UART0TX_Handler`;
- `drivers/systick.c` (**driver**): `systick_init`, `SysTick_Handler`, and `systick_uptime_ms`;
- `lib/format.c`: `vformat`, shared by both print functions;
- `kernel/kprintf.c`: `kprintf`, the kernel's polled print;
- `kernel/faults.c`: the four fault handlers that print `HFSR` or `CFSR`;
- `kernel/irq.s`: `irq_save` and `irq_restore`;
- `kernel/svc_entry.s` (**kernel entry**): the two-instruction `SVC_Handler`;
- `kernel/syscall.c` (**dispatcher and services**): the syscall implementations, `table`, and `svc_dispatch`;
- `lib/syscalls.s` and `lib/syscalls.h` (**user stubs**): `sys_write`, `sys_read`, and `sys_uptime` (`svc #n ; bx lr`) plus service numbers;
- `lib/printf.c` (**user library**): `write_all`, `printf`, and `sleep_ms`;
- `src/main.c` (**kernel boot**): initializes hardware, drops privilege, and calls `app_main`;
- `src/app.c` (**application**): `app_main`, the unprivileged user task.

Every file in `solution/` is commented line by line. If these notes and the code ever disagree, the code is the reference.

1.17 A prova di coglione

1.17.1 Unità 1

L'Unità 1 descrive la transizione da un programma dipendente dall'emulatore a un sistema operativo bare-metal capace di gestire autonomamente l'hardware. Il focus principale è la creazione di un driver per la porta seriale (UART) comunicando direttamente con i registri fisici. Architettura e Memory-Mapped I/O

Il sistema utilizza una CPU Cortex-M3 collegata a un bus veloce AHB per la memoria e a un bus secondario APB per le periferiche (come l'UART e i timer). Attraverso il paradigma Memory-Mapped I/O (MMIO), i dispositivi hardware vengono controllati leggendo e scrivendo su specifici indirizzi di memoria (ad esempio 0x40004000 per l'UART0). L'accesso a questi registri in C richiede rigorosamente la parola chiave volatile per forzare il compilatore a eseguire ogni singola operazione di I/O, evitando che vengano rimosse durante la fase di ottimizzazione. Sviluppo del Device Driver in Polling

Il software comunica con l'UART tramite polling: la CPU interroga continuamente un registro di stato per verificare se l'hardware è pronto a ricevere o inviare un dato, bloccandosi in un ciclo di attesa. Questa interazione deve seguire un preciso protocollo di stato (configurazione, abilitazione, check, trasferimento e reset) e deve evitare le trappole hardware del Read-Modify-Write, in particolare sovrascritture accidentali sui registri Write-1-to-Clear (W1C). Stampa Kernel e Gestione Errori

Il driver viene infine separato dalla logica di formattazione delle stringhe tramite la funzione vformat e i puntatori a funzione, dando vita a kprintf. Questa funzione permette al kernel (che opera in modalità privilegiata) di stampare messaggi formattati scavalcando le system call. Poiché non si basa sugli interrupt, kprintf è sicura da usare ovunque, permettendo ai gestori delle eccezioni (Fault Handlers) di stampare a terminale il contenuto dei registri diagnostici come il CFSR e l'HFSR in caso di crash di sistema.

Come leggere la tabella. Seconda colonna: l'idea in parole semplici. Terza colonna: **Esempio:** un caso concreto, **Attenzione:** l'errore tipico, **Regola:** la regola da ricordare.

Table 1.1: Riepilogo dei concetti dell'Unità 1

Concetto	In parole semplici	Esempio, errore tipico, regola
1. Hardware e scheda		
CMSDK e CMSIS	CMSDK è **hardware**: i blocchi già pronti (UART, timer, GPIO) che il costruttore del chip mette accanto alla CPU. CMSIS è **software**: uno stile standard di header C, con nomi e funzioni.	**Esempio:** `cmsdk.h` descrive l'hardware CMSDK, ma è scritto a mano con i nomi nello stile CMSIS, senza librerie dei produttori. **Regola:** CMSDK = circuiti, CMSIS = codice.

Concetto	In parole semplici	Esempio, errore tipico, regola
Scheda MPS2-AN385	La scheda che emuliamo: una CPU Cortex-M3 circondata da periferiche CMSDK. QEMU la simula con `mps2-an385`. Tutto è guidato da un unico clock a 25 MHz.	**Esempio:** velocità della UART e periodi dei timer si calcolano partendo da 25 MHz.
Bus AHB e APB *Advanced High-performance / Peripheral Bus*	Il bus è l'insieme di fili su cui viaggiano le load e le store. AHB è l'autostrada: veloce, per RAM, codice e GPIO. APB è la strada secondaria: lenta e a basso consumo, per UART, timer e watchdog. Un *bridge* (il casello) le collega.	**Regola:** dal codice C non cambia nulla: un registro è sempre solo un indirizzo. **Attenzione:** l'accesso a un registro APB costa qualche ciclo in più.
Linee IRQ *Interrupt Request*	Una periferica avvisa la CPU di un evento (un byte arrivato, un timer scaduto) alzando la sua linea IRQ. La linea numero n usa lo slot $16 + n$ della tabella dei vettori.	**Esempio:** UART0 RX = IRQ 0 → slot 16, UART0 TX = IRQ 1, TIMER0/1 = IRQ 8/9, dual timer = IRQ 10. Il watchdog usa l'NMI (slot 2). **Regola:** in questa unità gli interrupt sono spenti: la CPU chiede di continuo ai device (*polling*).
Mappa di QEMU	Con `info mtree -f` chiedi a QEMU cosa emula. Tre casi: modello reale (`prio 0`), funziona davvero; placeholder (`prio -1000`), finto: scritture ignorate, letture a 0; indirizzo vuoto, ignorato.	**Esempio:** UART0 e FPGAIO sono reali, GPIO0 è un placeholder. **Attenzione:** nessun caso dà un fault: se il codice non funziona, l'accesso a un device finto passa inosservato. Con `-d unimp` QEMU lo scrive a terminale.
Altre interfacce I/O	Oltre alla UART: GPIO (pin letti o pilotati a 0/1, per LED e pulsanti), SPI (clock condiviso e chip select, per flash e display), I2C (2 fili, indirizzi a 7 bit, per sensori), ADC/DAC (tensione ↔ numero).	**Regola:** si programmano come la UART: registri mappati in memoria, con solo più passi per ogni trasferimento.
DMA *Direct Memory Access*	Un controller che copia i dati da solo. Gli dai sorgente, destinazione e numero di byte, lo avvii e la CPU fa altro. Quando ha finito, avvisa con un interrupt.	**Attenzione:** CPU e DMA condividono il bus: non toccare il buffer mentre il DMA lo riempie. **Esempio:** nell'AN385 il DMA non c'è: la CPU sposta ogni byte.

Concetto	In parole semplici	Esempio, errore tipico, regola
2. Accesso ai registri		
I/O mappato in memoria *MMIO*	Ogni registro di una periferica ha un indirizzo fisso e lo usi con normali load e store. Nel C sembra una variabile, ma dietro c'è un dispositivo.	**Attenzione:** un registro non è RAM. Leggere può avere un effetto (`DATA` estrae il byte), scrivere può avviare un'azione, il valore può cambiare da solo (un bit di stato) e ciò che leggi può differire da ciò che hai scritto. **Regola:** segui sempre il manuale del device.
Struct sovrapposta	Una `struct` elenca i registri uno dopo l'altro e viene *sovrapposta* agli indirizzi veri: il primo campo sta all'indirizzo base, ogni `uint32_t` successivo 4 byte dopo.	**Esempio:** `CMSDK_UART0` è `0x40004000` visto come puntatore alla struct, quindi `UART0->STATE` è una load a `0x40004004`. **Regola:** `_Static_assert` con `offsetof` controlla gli offset in compilazione: un errore nel layout diventa errore di build.
`volatile`	Dice al compilatore: ogni lettura e ogni scrittura che vedi nel sorgente deve avvenire davvero, nell'ordine scritto. Senza, il compilatore crede che solo il tuo programma cambi i valori e toglie le letture che gli sembrano ripetute.	**Esempio:** a `-O2`, senza `volatile`, il ciclo `while (STATE & TXBF)` legge una volta sola e poi gira su se stesso (`b.n c`). **Attenzione:** a `-O0` il bug non si vede. `volatile` non rende l'accesso atomico e non protegge dati condivisi. **Regola:** sui registri, sempre `volatile`.
Read-modify-write	Scrivere `REG |= bit` sembra un'operazione sola, ma sono tre: leggi il registro, modifichi il bit nella CPU, riscrivi tutto. I tre passi non sono atomici: nel mezzo può succedere altro.	**Attenzione:** se un interrupt o il device cambia un altro bit tra lettura e scrittura, la tua scrittura rimette il valore vecchio e quel cambiamento si perde.
Registri W1C *Write-1-to-Clear*	Registri di stato in cui scrivere 1 su un bit cancella quel flag, mentre scrivere 0 lo lascia com'è.	**Esempio:** per cancellare solo il flag del bit 1: `STATUS = 1u << 1`. **Attenzione:** con `|=` rileggi tutti i flag attivi (come 1) e li riscrivi: li cancelli tutti.

Concetto	In parole semplici	Esempio, errore tipico, regola
Soluzioni sicure	Tre modi per evitare la trappola. (1) Registri SET/CLEAR: una sola scrittura cambia solo i bit scritti a 1. (2) Sui W1C, assegnazione `=` con il solo bit voluto. (3) Se serve davvero un read-modify-write, nessun altro deve toccare il registro nel frattempo.	**Esempio:** `OUTENABLESET = 1u << 3;`. Un RMW inevitabile è il CCR della settimana 2, dove STKALIGN doveva restare a 1. **Attenzione:** scrivi `1u << n` e non `1 << n`: l'`1` semplice è un `int` con segno e spostarlo nel bit 31 è comportamento indefinito.
3. La UART		
Semihosting contro UART	Il semihosting è una scorciatoia: la CPU si ferma e chiede al computer host, tramite il debugger, di stampare. Un sistema bare-metal vero non può contare sull'host e usa l'hardware: la UART.	**Regola:** da qui in poi si stampa scrivendo nei registri della UART, senza l'aiuto dell'host.
UART *Universal Asynchronous Receiver-Transmitter*	La porta seriale: un filo per trasmettere (TX) e uno per ricevere (RX). È asincrona: non c'è un clock condiviso, quindi i due lati si accordano sulla velocità (*baud rate*).	**Esempio:** QEMU collega UART0 al terminale con `-serial stdio`: un byte scritto in `DATA` appare a schermo, un tasto premuto arriva su RX.
`DATA` *offset 0x00*	La finestra dei dati. Se scrivi un byte, parte l'invio. Se leggi, ottieni il byte ricevuto.	**Attenzione:** leggere `DATA` ha un effetto: porta `RXBF` a 0, perché il byte è stato preso.
`STATE` *offset 0x04*	Le spie di stato. Bit 0 `TXBF` (TX Buffer Full): l'invio è occupato. Bit 1 `RXBF` (RX Buffer Full): c'è un byte da leggere. Bit 2 `TXOR` e bit 3 `RXOR`: overrun, cioè un byte perso.	**Regola:** guarda `STATE` *prima* di toccare `DATA`. **Attenzione:** i bit 2–3 sono W1C.
`CTRL` *offset 0x08*	Gli interruttori. Bit 0 `TXEN`: trasmettitore acceso. Bit 1 `RXEN`: ricevitore acceso. Bit 2–3 (`TXIRQEN`, `RXIRQEN`): interrupt di TX e RX. Bit 4–5: interrupt di overrun.	**Attenzione:** dopo il reset `CTRL` vale 0: TX e RX sono spenti e la UART ignora ciò che scrivi in `DATA`. Con `-d guest_errors` QEMU segnala l'errore.
`INTSTATUS` *offset 0x0C*	Dice quale interrupt è in attesa: bit 0 TX, bit 1 RX, bit 2 TXOR, bit 3 RXOR.	**Regola:** è W1C: per confermare un evento scrivi 1 sul suo bit.

Concetto	In parole semplici	Esempio, errore tipico, regola
`BAUDDIV` *offset 0x10*	Il divisore che fissa la velocità: clock diviso baud rate, almeno 16.	**Esempio:** 25 000 000/115 200 ≈ 217. Nel nostro codice scriviamo 16 perché QEMU ignora la velocità.
Un byte per direzione	La UART ha un solo posto per direzione. In TX, `TXBF` = 1 significa che l'ultimo byte sta ancora uscendo. In RX, `RXBF` = 1 significa che un byte aspetta di essere letto.	**Attenzione:** scrivere con `TXBF` = 1 perde il nuovo byte (`TXOR`). Se in RX arriva un altro byte prima che tu legga, va perso (`RXOR`).
4. Il driver		
Protocollo del device	Un device non è una variabile ma un protocollo: passi in ordine fisso sui registri. (1) Configura. (2) Abilita. (3) Controlla lo stato. (4) Muovi i dati. (5) Conferma l'evento.	**Esempio:** sulla UART: `BAUDDIV` → `CTRL` → `STATE` → `DATA` → `INTSTATUS`. **Regola:** prima configura e poi abilita, prima lo stato e poi i dati, e conferma ogni evento cancellando il suo flag.
Driver e API *Application Programming Interface*	Il driver è il codice che conosce il protocollo e lo esegue una volta sola, offrendo al resto del sistema funzioni con un significato ("manda questo byte") invece di indirizzi e bit.	**Regola:** chi usa il driver include `drivers/uart.h` e non `cmsdk.h`: gli indirizzi restano nascosti dentro il driver. *Polled* = il driver legge `STATE` in un ciclo, senza interrupt.
`uart_init`	Si chiama una sola volta all'avvio. Configura e abilita: scrive `BAUDDIV = 16` e poi `CTRL = TXEN | RXEN`.	**Regola:** le due maschere si combinano con `|` per accendere TX e RX insieme. **Attenzione:** senza `uart_init` la UART scarta ogni byte (`CTRL` = 0).
`uart_putc_polled`	Invia un byte: aspetta finché `TXBF` vale 0 (c'è posto), poi scrive il byte in `DATA`.	**Esempio:** `uart_putc_polled('A')`. **Attenzione:** doppio cast `(uint32_t)(unsigned char)c`: un `char` con segno come `0xE9` diventerebbe `0xFFFFFFE9`. Passando per `unsigned char` resta `0x000000E9`.
`uart_getc_polled`	Prova a leggere un byte: se `RXBF` vale 0 restituisce −1 subito, senza aspettare. Altrimenti restituisce il byte letto da `DATA`.	**Esempio:** nel ciclo di echo, se il valore è ≥ 0 lo rimanda indietro. **Attenzione:** la CPU gira milioni di volte al secondo anche se non premi nulla: è il prezzo del polling.

Concetto	In parole semplici	Esempio, errore tipico, regola
Driver con puntatori a funzione	I sistemi grandi descrivono ogni driver con una `struct` di puntatori a funzione (`init`, `read`, `write`): il codice generico usa qualsiasi device senza conoscerlo.	**Esempio:** Linux usa `file_operations`. **Regola:** questa settimana non la scriviamo: c'è una sola console, quindi si chiamano direttamente le funzioni della UART.
5. kprintf e fault		
Funzioni variadiche	Funzioni con un numero qualsiasi di argomenti, dichiarate con `...` come `printf`. Il cursore `va_list ap` scorre gli argomenti extra: `va_start` lo mette dopo l'ultimo argomento fisso, `va_arg(ap, tipo)` legge un valore e avanza, `va_end` chiude.	**Attenzione:** la funzione non sa né quanti argomenti riceve né di che tipo: lo dice la stringa di formato. Se il tipo non corrisponde, il comportamento è indefinito. **Regola:** un `char` arriva come `int`, quindi `%c` legge con `va_arg(ap, int)`.
`vformat` e `out`	`vformat(out, ctx, fmt, ap)` trasforma il formato in caratteri e chiama `out(c, ctx)` per ognuno: sa *cosa* stampare, non *dove*. `out` è un puntatore a funzione (la destinazione), `ctx` un puntatore opaco che passa senza guardarlo.	**Regola:** per un'altra destinazione (schermo, buffer, rete) basta un altro `out`: il formato si scrive e si prova una volta sola. **Esempio:** per la UART `ctx` vale 0.
Conversioni supportate	Dopo il `%` la lettera dice cosa stampare: `%c` carattere, `%s` stringa, `%d` intero con segno, `%u` intero senza segno, `%x` esadecimale, `%%` il simbolo %.	**Attenzione:** niente larghezza né precisione, `%x` non aggiunge `0x`, `%s` nullo stampa `(null)`, una conversione sconosciuta esce com'è. **Regola:** per `%d` si usa `0u - (uint32_t)v` e non `-v`: con `INT_MIN` il `-v` è un overflow.
`kprintf` k = *kernel*	Collega il formato alla UART: `va_start`, poi `vformat(kprintf_out, 0, fmt, ap)`, poi `va_end`. `kprintf_out` manda ogni carattere a `uart_putc_polled`.	**Esempio:** `kprintf("d=%d\n", -42)` fa 6 chiamate a `out`: `d`, `=`, `-`, `4`, `2` e l'a capo. **Regola:** è in polling, quindi funziona ovunque, anche dentro un HardFault. `(void) ctx` evita il warning sul parametro non usato.
`kprintf` e `printf`	Il kernel è privilegiato e parla direttamente col driver. Un'applicazione utente non può toccare l'hardware: il suo `printf` chiede al kernel di stampare con una system call.	**Attenzione:** il kernel non può usare `printf` per i propri messaggi: una `svc` dentro l'handler di SVCall causa un HardFault.

Concetto	In parole semplici	Esempio, errore tipico, regola
Fault che stampano	Gli handler dei fault ora scrivono il motivo prima di fermarsi. Due registri lo dicono: `CFSR` (`0xE000ED28`) per MemManage, BusFault e UsageFault; `HFSR` (`0xE000ED2C`) per l'HardFault, dove il bit 30 `FORCED` indica un fault escalato.	**Esempio:** `10u / zero` dà UsageFault con `CFSR=0x2000000`: il bit 25 è `DIVBYZERO`, perché il CCR abilita la trappola. **Regola:** `zero` è `volatile`, così la divisione avviene a runtime e non in compilazione.
6. Orientarsi nel progetto		
Struttura delle cartelle	Il codice è diviso per ruolo: `include/` (mappa dei registri `cmsdk.h`), `drivers/` (UART), `kernel/` (`kprintf`, fault), `lib/` (`vformat`, conversione dei numeri), `src/` (`main`).	**Regola:** all'esame di solito ricevi un progetto già fatto e devi aggiungere una funzione o correggere un bug: saper trovare il file giusto conta quanto conoscere i registri.

1.17.2 Unità 2

Questa unità segna il passaggio fondamentale dal **polling** (in cui la CPU spreca tempo aspettando in un ciclo infinito) agli **interrupt**, meccanismi che permettono alla CPU di lavorare o riposare finché l'hardware non richiede attenzione.

- **Regole di convivenza (AAPCS):** Per far comunicare il C e l'Assembly, si usa lo standard AAPCS. Definisce quali registri possono essere sovrascritti liberamente da una funzione (*caller-saved*) e quali devono essere rigorosamente salvati e ripristinati (*callee-saved*).

- **Gestione hardware delle eccezioni:** Quando scatta un interrupt, l'hardware Cortex-M salva in automatico 8 registri (che corrispondono esattamente ai *caller-saved*) nello stack. Nel Link Register viene inserito un codice magico (`EXC_RETURN`) che dice alla CPU come ripristinare il contesto a fine funzione. Questo design geniale permette di scrivere gli handler degli interrupt come normalissime funzioni C.

- **Controllo Globale e SysTick:** Il registro **PRIMASK** funge da interruttore generale degli interrupt. Il **SysTick Timer** è invece il cuore del sistema operativo: scatta a intervalli regolari (es. ogni 10 ms) garantendo la *prelazione hardware* (ferma i programmi bloccati in loop), aggiorna il tempo di attività del sistema (gestendo i rischi di overflow) e permette alla CPU di risparmiare energia mettendosi in pausa con l'istruzione `wfi` (Wait For Interrupt).

- **Gestione delle Periferiche (NVIC):** Gli interrupt esterni sono gestiti dal controller NVIC. Affinché un interrupt arrivi alla CPU deve superare i "3 cancelli" (abilitazione sul dispositivo, sul NVIC e globalmente). Vige una **Regola d'Oro**: l'handler deve sempre confermare l'interrupt pulendo il flag hardware, altrimenti l'eccezione scatterà all'infinito bloccando il sistema.

- **Concorrenza e Sezioni Critiche:** Gli interrupt causano problemi di concorrenza (le *Race Condition*): un interrupt può interrompere il `main` a metà di un calcolo, corrompendo i dati condivisi (*lost update*). Si risolve creando **Sezioni Critiche** tramite le funzioni `irq_save` e `irq_restore`, che disabilitano brevissimamente gli interrupt garantendo l'esecuzione atomica. L'uso del ripristino (*restore*) garantisce che gli interrupt non vengano riabilitati per sbaglio se le funzioni protette sono annidate.
- **Il Ring Buffer e l'UART:** Per inviare lunghi testi senza bloccare il sistema, si introduce un modello Produttore/Consumatore usando un **Ring Buffer** (una coda circolare di 64 byte). Il `main` (Produttore) salva i dati nel buffer e, grazie alla *Priming Rule*, innesca l'hardware inviando manualmente il primo byte. Da quel momento, l'handler hardware (Consumatore) si sveglia in automatico a ogni byte inviato e svuota il buffer in background, mentre il programma principale può letteralmente "dormire".

Come leggere la tabella. Seconda colonna: l'idea in parole semplici. Terza colonna: **Esempio:** un caso concreto, **Attenzione:** l'errore tipico, **Regola:** la regola da ricordare.

Table 1.2: Riepilogo dei concetti dell'Unità 2

Concetto	In parole semplici	Esempio, errore tipico, regola
1. Polling, interrupt e convenzione di chiamata		
Polling vs interrupt	*Polling*: la CPU legge di continuo il registro `STATE` del device finché è pronto. *Interrupt*: il device manda un segnale solo quando serve. La CPU si ferma, esegue l'*handler* e riprende esattamente da dove era.	**Esempio:** 10 bit a 115 200 baud durano $\approx 87\ \mu$s, cioè **2 170 cicli** a 25 MHz, tutti sprecati nel polling. **Regola:** latenza: polling fino a 1 iterazione, interrupt $\sim$12 cicli (Cortex-M3), ma servono handler, protezione dei dati e priorità. **Regola:** `kprintf` resta in polling: deve stampare anche in `HardFault`, dove gli interrupt normali sono bloccati.
AAPCS *Procedure Call Standard for the Arm Architecture*	Le regole con cui C e Assembly si passano i dati. Ogni registro ha un ruolo preciso: chi lo può sovrascrivere e chi deve rimetterlo a posto.	**Regola:** `r0-r3`: parametri (`r0` anche valore di ritorno), **caller-saved**. `r12`: scratch, caller-saved. **Regola:** `r4-r11`: **callee-saved**, se li usi fai `push` e `pop`. **Regola:** `lr`: indirizzo di ritorno, se chiami un'altra funzione salvalo prima. `sp`: sempre multiplo di 8.

Concetto	In parole semplici	Esempio, errore tipico, regola
2. Gestire gli interrupt da Assembly		
PRIMASK	Interruttore generale a 1 bit della CPU: `1` = interrupt bloccati, `0` = CPU in ascolto. Il C non può toccare i registri interni, servono istruzioni Assembly.	**Esempio:** `mrs r0, primask` legge, `msr primask, r0` scrive, `cpsid i` / `cpsie i` disabilita / abilita gli interrupt globali.
`irq_save` e `irq_restore`	Servono a proteggere un pezzo di codice, anche quando le protezioni sono annidate. Prima si salva lo stato di PRIMASK, poi lo si ripristina.	**Esempio:** `irq_save()` mette il vecchio PRIMASK in `r0` e maschera. `irq_restore(old)` lo rimette. Il test stampa `0`. In GDB, dopo `finish`: `r0 = 0x0`, PRIMASK = 1. **Attenzione:** `cpsid` e `cpsie` nudi: la funzione interna riabilita gli interrupt troppo presto e distrugge la protezione di quella esterna. **Regola:** usano solo `r0` e `lr` (caller-saved): niente `push` e `pop`.
3. Entrata e uscita da un'eccezione		
Stacking automatico	Prima della prima istruzione del handler, l'hardware fa tre cose: (1) mette 8 word (il *frame*) sullo stack, (2) scrive `EXC_RETURN` in `lr`, (3) legge l'indirizzo dalla vector table e salta, in Handler mode.	**Esempio:** il frame, da `sp+0` a `sp+28`: `r0, r1, r2, r3, r12, LR, PC, xPSR`. **Regola:** sono esattamente i registri caller-saved dell'AAPCS: l'handler è una **normale funzione C** (es. `void SysTick_Handler(void)`). Lo stack resta allineato a 8 byte (`STKALIGN`).
`EXC_RETURN`	Dentro il handler, `lr` non contiene un vero indirizzo ma un valore `0xFFFFFF..`, in una zona senza codice. Con `bx lr` la CPU lo riconosce, toglie il frame dallo stack e riprende dal `PC` salvato.	**Esempio:** `0xFFFFFFF9`: Thread mode, Main stack (l'unico usato per ora). `0xFFFFFFF1`: Handler mode, Main stack. `0xFFFFFFFD`: Thread mode, Process stack. **Regola:** il handler C finisce con un `bx lr` ordinario: la magia è tutta nel valore di `lr`.

Concetto	In parole semplici	Esempio, errore tipico, regola
Tail-chaining e late arrival	Due ottimizzazioni hardware per eccezioni vicine, senza intervento del software.	**Esempio:** *tail-chaining*: se un'eccezione è pendente quando un handler finisce, si salta direttamente al suo handler senza fare pop e push del frame (∼6 cicli invece di 24). **Esempio:** *late arrival*: se durante lo stacking arriva un'eccezione più urgente, si usa il frame già in costruzione, si esegue prima quella, poi si fa tail-chain. **Regola:** in entrambi i casi un solo frame descrive il codice originale interrotto.
4. SysTick: il battito del sistema		
SysTick	Timer a 24 bit che conta alla rovescia, presente in ogni Cortex-M agli stessi indirizzi. Ogni volta che arriva a 0 genera un'eccezione: è la **preemption hardware**, che ridà il controllo al kernel anche se un programma va in loop infinito.	**Esempio:** `CTRL 0xE000E010` (bit 0 ENABLE, bit 1 TICKINT, bit 2 CLKSOURCE, bit 16 COUNTFLAG), `LOAD 0xE000E014`, `VAL 0xE000E018`, `CALIB 0xE000E01C`. **Esempio:** avvio: `LOAD`, `VAL = 0`, `CTRL = 7`. **Regola:** periodo = LOAD + 1 cicli. Priorità `0xE0` (la meno urgente). Nessun acknowledge: il pending si cancella da solo.
Scelta del tick	**LOAD = f / rate − 1**. A 25 MHz e 100 Hz: 25 000 000/100 − 1 = 249 999 (`0x3D08F`), che entra nei 24 bit. Tick veloce: più precisione ma più overhead. Tick lento: meno overhead ma meno risoluzione.	**Esempio:** un tick da 1 ms costa 1 000 entrate e uscite al secondo anche a riposo, e in QEMU va ∼26% più lento del tempo reale. **Regola:** il kernel usa **10 ms (100 Hz)**.
Uptime e wraparound	Il handler somma `TICK_MS` a `uptime_ms`, che è `volatile` perché cambia in modo asincrono. La lettura è un solo accesso a 32 bit, quindi atomica. Dopo ∼49,7 giorni il contatore riparte da 0.	**Esempio:** `start = 0xFFFFFF00`, `now = 0x64`: `now - start = 0x164` (356 ms), grazie all'aritmetica modulo 2^{32}. **Regola:** misura sempre il tempo con `now - start >= delay`, in unsigned.

Concetto	In parole semplici	Esempio, errore tipico, regola
`wfi` *Wait For Interrupt*	Istruzione che mette la CPU in sleep finché non c'è un'eccezione pendente.	**Regola:** un idle loop con `wfi` dentro un `for(;;)` gira una volta per interrupt, invece di milioni di controlli inutili: risparmia energia e resta reattivo.
5. NVIC e vector table		
NVIC *Nested Vectored Interrupt Controller*	Il portiere tra le periferiche esterne e la CPU: gestisce abilitazione e priorità delle 32 linee (IRQ 0–31). SysTick non passa di qui: è un'eccezione interna al core.	**Esempio:** UART0 RX = IRQ 0, UART0 TX = IRQ 1. L'IRQ n è l'eccezione $16+n$, con vettore a offset $4 \times (16+n)$: UART0 TX → `0x44`. **Esempio:** `ISER 0xE000E100` abilita, `ICER 0xE000E180` disabilita, `ISPR 0xE000E200` forza il pending, `IPR 0xE000E400` imposta la priorità (valore basso = più urgente). **Regola:** scrivere 1 agisce, scrivere 0 non fa nulla: si scrive direttamente, senza read-modify-write.
I tre gate	Un interrupt arriva alla CPU solo se supera tre condizioni indipendenti.	**Regola:** **Device**: periferica configurata (es. `TXIRQEN` in `CTRL`). **NVIC**: linea abilitata con `ISER`. **CPU**: PRIMASK = 0 e priorità *strettamente* maggiore del codice in esecuzione. **Attenzione:** se manca il gate 1 o il 2, l'interrupt è ignorato in silenzio: stesso sintomo.
Regola d'oro: acknowledge	Le linee NVIC sono *level-sensitive*: il device tiene la linea alta finché il suo flag resta acceso.	**Regola:** il handler deve **cancellare il flag nel device** prima di tornare. **Attenzione:** con `bx lr` senza ack, l'interrupt riscatta subito, all'infinito. **Esempio:** `INTSTATUS` è write-1-to-clear: `CMSDK_UART0->INTSTATUS = UART_INT_TX;` cancella il bit 0 e lascia gli altri. Se più device condividono una linea, il handler controlla lo stato di tutti.

Concetto	In parole semplici	Esempio, errore tipico, regola
Vector table	16 slot del core + 32 IRQ = 48 word = 192 byte (`0xC0`). Gli slot inutilizzati puntano a `Default_Handler`.	**Esempio:** `UART0TX_Handler` = IRQ 1 (word 17), `TIMER0_Handler` = IRQ 8 (word 24). In GDB: `x/4wx 0x40`. **Regola:** i nomi sono `.weak` + `.thumb_set`: una funzione C con lo stesso nome sostituisce il default.
6. Concorrenza e dati condivisi		
Race condition	Con gli interrupt, `main` può essere fermato in qualsiasi istante, anche a metà di una riga C. `count++` sono tre istruzioni: `ldr`, `adds`, `str`.	**Esempio:** `main` legge 5, scatta l'interrupt che scrive 6, `main` riprende con il vecchio 5 e scrive 6: il lavoro del handler è perso (*lost update*).
Sezione critica	Piccolo blocco in cui la CPU ignora gli interrupt, racchiuso tra `irq_save()` e `irq_restore()`. Un interrupt in arrivo resta pendente fino al ripristino.	**Regola:** *asimmetria*: solo `main` si protegge, il handler non può essere interrotto da `main`. **Regola:** *brevità*: niente delay o loop dentro. **Regola:** *nesting*: chiudi sempre con `irq_restore(s)`, mai con un enable grezzo.
Ring buffer	Array circolare in cui `main` (producer) deposita i caratteri e il handler (consumer) li preleva. `head` è scritto solo da `main`, `tail` solo dal handler, entrambi `volatile`.	**Esempio:** `TX_BUF_SIZE = 64`. Indice successivo: `(i+1) & (N-1)`. Vuoto: `head == tail`. Pieno: `((head+1) & (N-1)) == tail`. **Regola:** dimensione potenza di 2: l'`&` sostituisce il modulo. Si usano **63 slot su 64**, per non confondere pieno e vuoto.
7. Driver UART a interrupt		
`uart_write` (producer)	Dentro una sezione critica copia nel buffer quanti caratteri riesce e restituisce quanti ne ha accettati. Se l'UART è ferma non genera interrupt, quindi va avviata a mano.	**Regola: Priming Rule**: se `!tx_busy && !tx_empty`, abilita `TXIRQEN`, attende `TXBF` libero, scrive il primo byte e imposta `tx_busy = 1`. Senza il primo byte non scatta nessun interrupt.
`UART0TX_Handler` (consumer)	Scatta da solo ogni volta che un byte lascia l'hardware. Non serve `irq_save`: `main` non può interromperlo.	**Regola:** (1) ack: `INTSTATUS = UART_INT_TX`. (2) Se il buffer non è vuoto, invia il byte successivo. (3) Altrimenti spegne `TXIRQEN` e imposta `tx_busy = 0`.

Concetto	In parole semplici	Esempio, errore tipico, regola
`uart_init`	Azzera lo stato del buffer e apre il gate NVIC.	**Esempio:** `head = tail = tx_busy = 0`. `NVIC->IPR[UART0TX_IRQn] = 0xC0` (meno urgente delle System Call, più urgente di SysTick). Abilita l'IRQ 1 con `ISER`.
Test dei 160 byte	Il messaggio (4 righe da 40 byte) è più grande del buffer. Se `uart_write` accetta 0 byte, `main` fa `wfi` e il handler lo risveglia quando libera spazio.	**Esempio:** `uart_write` gira 3 volte (63 + 63 + 34 byte), il handler scatta 160 volte e `main` dorme quasi sempre.
8. Dove siamo arrivati		
Stato a fine Unità 2	Driver e kernel completi (`uart.c`, `systick.c`, `irq.s`, `startup.s`), ma tutto gira ancora in modalità privileged dentro `main`.	**Esempio:** `main` stampa il banner, avvia SysTick, invia i 160 byte, poi stampa l'uptime ogni secondo. **Regola:** Unità 3: ambiente unprivileged e confine sicuro tra kernel e applicazione.

1.17.3 Unità 3

L'idea di fondo

Fino alla Unit 2 tutto girava in modalità privilegiata dentro `main`. Ora si costruisce un vero confine: l'applicazione gira **non privilegiata** e può raggiungere l'hardware solo *chiedendo* al kernel, tramite le **system call**, costruite sull'eccezione `svc`.

Dalla Week 2 ci portiamo quattro fatti:

- `main` parte privilegiato. Con `CONTROL.nPRIV = 1` si passa al Thread mode non privilegiato, e il passaggio è a senso unico, perché modificare `CONTROL` è a sua volta un'operazione privilegiata.
- Il codice non privilegiato che tocca lo System Control Space (SysTick, NVIC, SCB) causa un **BusFault**.
- `svc #n` è sempre permessa e genera l'eccezione **SVCall**.
- Ogni handler gira in Handler mode (privilegiato). Al ritorno la CPU riparte subito dopo la `svc` e torna non privilegiata.

Quindi `svc` è l'unica porta controllata verso il codice privilegiato: l'app bussa, il kernel decide. In Week 2 `SVC_Handler` faceva un solo lavoro fisso; ora diventa un'interfaccia che smista più servizi, riceve argomenti e restituisce risultati.

La regola: l'app chiede, il kernel fa

L'app non chiama mai un driver e non tocca mai un registro hardware. Ogni I/O attraversa il confine con una system call, come in Linux, dove `printf` finisce in `write`. Il percorso completo è:

`app_main` → `printf` → `write_all` → `sys_write` → `svc #1` → **confine** → `SVC_Handler` → `svc_dispatch` → `table[n]` → `uart_write` + `UART0TX_Handler` → registri UART0 a `0x40004000`.

Sopra il confine si è in Thread mode non privilegiato, sotto in Handler mode privilegiato. I file coinvolti sono `src/app.c`, `lib/printf.c`, `lib/syscalls.s`, `kernel/svc_entry.s`, `kernel/syscall.c` e `drivers/uart.c`.

Protezione logica e fisica. Senza MPU l'hardware blocca solo l'accesso allo System Control Space (protezione *fisica*). I registri UART restano raggiungibili dall'utente, quindi tenerli lontani è solo una regola *logica* del nostro design. Il confine però è necessario, perché il driver UART usa `irq_save()` (cioè `cpsid i`) per proteggere il ring buffer. In modalità non privilegiata `cpsid` viene ignorata in silenzio: il driver crederebbe di essere protetto con gli interrupt ancora attivi.

ABI e servizi

Un'API definisce le regole a livello di codice C. Un'**ABI** è il contratto a basso livello tra compilatore e hardware, fatto di registri e istruzioni, perché dopo la compilazione i nomi spariscono. Per le system call dice:

- il **numero del servizio** è dentro l'istruzione `svc #n`;
- gli **argomenti** vanno in `r0`, `r1`, `r2` (AAPCS);
- il **risultato** torna in `r0`, dove il chiamante C lo cerca.

Per il programmatore una syscall sembra una normale funzione C, ma sotto scatta una `svc` invece di una `bl`. Le costanti (`SYS_WRITE`, `SYS_READ`, `SYS_UPTIME`) stanno in `lib/syscalls.h`, condiviso da utente e kernel. I servizi sono:

- **1,** `sys_write(buf, len)`**:** restituisce i byte accettati (da 0 a `len`) e passa da `uart_write`.
- **2,** `sys_read()`**:** restituisce un byte oppure `-1` se non c'è nulla, via `uart_getc_polled`.
- **3,** `sys_uptime()`**:** restituisce i ms dal boot, via `systick_uptime_ms`.
- **0 e** ≥ 4**:** servizio sconosciuto, restituiscono `-1`.

Nessun servizio aspetta: `sys_write` prende quello che entra nel ring buffer TX e torna subito. L'attesa spetta al chiamante non privilegiato.

Stub ed Exception Frame

Lo **stub** è una minuscola funzione assembly in `lib/syscalls.s`: due istruzioni, `svc #n` e `bx lr`. Il viaggio di una chiamata, per esempio `sys_write`, è questo:

1. **C prepara i dati:** mette gli argomenti in `r0` e `r1`.
2. **Lo stub esegue** `svc #1`**:** l'hardware sospende l'app e salva 8 registri sullo stack, formando l'**Exception Frame**.

3. **Il kernel lavora:** vede il frame come array `uint32_t *frame`, dove `frame[0]` è il vecchio `r0` e `frame[1]` il vecchio `r1`. Legge gli argomenti, scrive sulla UART e mette il risultato in `frame[0]`.

4. **L'hardware ripristina:** rimette il frame nei registri, quindi `r0` contiene la risposta del kernel.

5. **Lo stub fa `bx lr`:** C trova il valore in `r0` senza sapere nulla del passaggio nel kernel.

Il numero dopo `svc` è codificato nell'opcode, quindi deve essere un **letterale** (1, 2, 3). L'assembly non legge le macro C, e i numeri vanno tenuti allineati a mano.

Entry point e dispatcher

`SVC_Handler` (`kernel/svc_entry.s`) deve essere **in assembly**. Sono solo due istruzioni:

- `mrs r0, msp` mette in `r0` l'indirizzo del frame;
- `b svc_dispatch` salta al dispatcher in C.

In C il compilatore inserirebbe un prologo (es. `push {r7, lr}`) che sposta `sp`, e cambierebbe con `-O0` e `-O2`: perderemmo l'indirizzo del frame. Si usa `b` e non `bl`, perché `bl` sovrascriverebbe `lr` distruggendo `EXC_RETURN` (`0xFFFFFFF9`). Con `b`, quando `svc_dispatch` ritorna parte direttamente il ritorno da eccezione.

Estrarre il numero. In Thumb `svc #n` è un'istruzione a 16 bit: byte alto `0xDF` (opcode), byte basso `n`, quindi `svc #1` = `0xDF01`. Il Cortex-M3 è little-endian, perciò in memoria si trova prima `n` e poi `0xDF` (`01 DF`). Il frame è `frame[0..7]` = `r0-r3, r12, lr, pc, xPSR`. Il `PC` salvato (`frame[6]`) punta all'istruzione dopo la `svc`, quindi `0xDF` sta a `PC - 1` e `n` a `PC - 2`. Gli argomenti sono in `frame[0]`, `frame[1]`, `frame[2]`.

Dispatcher. In `kernel/syscall.c` c'è una tabella di puntatori a funzione con inizializzatori designati: l'indice è il numero del servizio, e l'indice 0 resta vuoto (`NULL`). `svc_dispatch` legge `n = ((uint8_t *) frame[6])[-2]`. Se `n >= TABLE_SIZE` o `table[n] == 0`, scrive `-1` in `frame[0]`. Altrimenti fa `frame[0] = table[n](frame[0], frame[1], frame[2])`. Il risultato va in `frame[0]` perché, al ritorno, l'hardware ripopola `r0` proprio da lì: lasciarlo nel `r0` fisico non servirebbe.

Prove.

- *Prima syscall:* in `main` si sostituisce `systick_uptime_ms()` con `sys_uptime()`. L'output è identico, ma il percorso passa da `svc #3`. `svc` funziona anche da privilegiato.
- *GDB:* con `break *SVC_Handler`, `x/8wx $sp` mostra il frame e `p/x $lr` dà `0xFFFFFFF9`. Con `x/2bx ((unsigned *)$sp)[6] - 2` si leggono `0x03` e `0xDF`, quindi `svc #3` e $n = 3$ (`SYS_UPTIME`). Con `stepi` due volte si entra in `svc_dispatch`, e `finish` torna al `bx lr` di `sys_uptime`.

Dove funziona svc e perché il kernel non aspetta

SVCall ha priorità `0x80`. Un'eccezione può interrompere solo codice **meno urgente** (valore più alto = meno urgente). Quindi `svc` viene accettata:

- da Thread mode (`app_main`, `main`);
- dentro `SysTick_Handler` (`0xE0`) e `UART0TX_Handler` (`0xC0`), dove SVCall si annida.

Va invece in **HardFault**:

- dentro `SVC_Handler` e ciò che chiama (`0x80`);
- in un fault handler (`0x00` o più urgente);
- con `PRIMASK = 1`, cioè tra `irq_save` e `irq_restore`.

Per questo il kernel tiene il suo `kprintf` a polling, indipendente dalle syscall.

Il kernel non deve mai aspettare dentro un handler. Se `sys_write_impl` ciclasse dentro SVCall (`0x80`) in attesa che il buffer TX si svuoti, l'interrupt UART TX (`0xC0`, meno urgente) non potrebbe mai partire e il buffer non si svuoterebbe mai: deadlock. Anche SysTick (`0xE0`) sarebbe bloccato e l'orologio si fermerebbe. Quindi il kernel accetta ciò che entra e torna subito, e l'attesa (`wfi`) si fa in Thread mode, dove ogni interrupt può girare.

Libreria utente

`printf` in `lib/printf.c` lavora in due passi: il motore condiviso `vformat` formatta in un buffer locale da 64 byte sullo stack utente, poi `sys_write` lo invia. Il confronto con `kprintf` è netto:

- `kprintf` è privilegiato, usa il driver in polling direttamente, aspetta in busy-wait e funziona anche dentro handler e HardFault, perché non usa `svc`.
- `printf` è non privilegiato, passa da `vformat` → `sys_write` → `svc` → ring buffer, aspetta con `wfi` e dentro un handler va in HardFault.

`write_all` cicla finché tutti i byte sono inviati: se `sys_write` restituisce 0, il buffer è pieno e fa `wfi` invece di consumare CPU. La CPU dorme finché un interrupt (es. UART TX che libera spazio) la risveglia.

`sleep_ms(ms)` salva `start = sys_uptime()` e, finché `sys_uptime() - start < ms`, fa `wfi`. La sottrazione unsigned regge il wraparound (∼49,7 giorni). SysTick scatta ogni 10 ms, quindi ogni `wfi` si risveglia almeno così spesso. Il kernel non viene mai bloccato.

Avvio e applicazione

`src/main.c` è l'inizializzatore del **kernel**: gira privilegiato una sola volta, in cinque passi.

1. `uart_init()`, per primo, così si può stampare.
2. `kprintf` del banner, in polling.
3. `systick_init()`, il tick da 10 ms.
4. **Privilege drop:** `CONTROL |= 1`, seguito da `isb`, che scarta le istruzioni già prefetchate sotto le vecchie regole.
5. `app_main()`, che non ritorna mai.

`app_main` (`src/app.c`) include solo `lib/syscalls.h` e `lib/printf.h`. Ogni 1000 ms stampa l'uptime, confrontando `now - last >= 1000` e facendo `last += 1000`, perché contare le iterazioni accumulerebbe deriva per il tempo speso in `printf`. A ogni giro controlla `sys_read()` (`-1` = nessun tasto), fa l'eco dei tasti e chiama `sleep_ms(10)`. Anche se l'app è non privilegiata, interrupt e SVCall girano privilegiati: l'hardware alza il privilegio all'ingresso dell'eccezione e lo riabbassa al ritorno.

Due esperimenti

1. `printf` dentro il kernel. Si aggiunge `#include "lib/printf.h"` e un `printf("uptime called\n")` in `sys_uptime_impl`. Invece di una stampa ogni 10 ms si ottiene `HardFault: HFSR=0x40000000`. La catena `printf` → `write_all` → `sys_write` → `svc #1` avviene dentro SVCall (`0x80`), e SVCall non è più urgente di sé stessa. Un interrupt potrebbe restare pendente, ma una `svc` no: va presa subito o mai, quindi la CPU la **escala** a HardFault. `HFSR` bit 30 (`FORCED`) significa "un fault è stato escalato". Il report si stampa comunque, perché `HardFault_Handler` usa `kprintf`, che non ha bisogno di `svc`. Si annullano le modifiche.

2. Kernel che aspetta. Si fa in modo che `sys_write_impl` cicli finché tutto è accettato e si aggiunge in `app_main` una `sys_write` da 130 byte (più dei 64 del buffer). L'output è un solo `0` e poi il silenzio.

- In GDB, `p $xpsr & 0x1ff` dà `11`, quindi la CPU è nell'handler SVCall.
- `bt` mostra `uart_write` ← `sys_write_impl` ← `svc_dispatch` ← `<signal handler called>` ← `sys_write` ← `app_main`, con in cima eventualmente `irq_save` o `irq_restore`.
- Cosa è successo: la prima `uart_write` riempie il buffer e "innesca" la UART col primo byte (lo `0` visibile). Il buffer resta pieno, ma solo `UART0TX_Handler` (`0xC0`) può svuotarlo, e la CPU è in SVCall (`0x80`, più urgente). L'interrupt resta pendente per sempre e SysTick si ferma.
- La soluzione è il design già adottato: il kernel non aspetta, ed è `write_all` ad attendere con `wfi` in Thread mode. Si annullano le modifiche.

Privilegio e livelli in GDB

Con `break main` e `break app_main`, `p/x $control` dà `0x0` in `main` (privilegiato) e `0x1` in `app_main` (`nPRIV` attivo). Dentro `sys_write_impl`, `$control` è ancora `0x1`: la CPU è privilegiata perché sta in **Handler mode**, non perché `CONTROL` sia cambiato. I 9 bit bassi di `xPSR` sono il numero dell'eccezione, e `0xb` (11) è SVCall.

Il backtrace mostra tutti i livelli: `sys_write_impl`, `svc_dispatch`, `<signal handler called>`, `sys_write`, `write_all` (con `len=34`), `printf`, `app_main`, `main`. Il frame `<signal handler called>` è il marcatore GDB dell'ingresso in un'eccezione: sopra gira il kernel privilegiato in Handler mode, sotto l'applicazione non privilegiata.

Come leggere la tabella. Seconda colonna: l'idea in parole semplici. Terza colonna: **Esempio:** un caso concreto, **Attenzione:** l'errore tipico, **Regola:** la regola da ricordare.

Table 1.3: Riepilogo dei concetti dell'Unità 3

Concetto	In parole semplici	Esempio, errore tipico, regola
1. Privilegio e confine del kernel		
Privilegio e `svc`	Il codice utente gira **non privilegiato** (`CONTROL.nPRIV = 1`), il kernel gira privilegiato. `svc` è l'unica porta controllata: l'app bussa, il kernel decide.	**Regola:** il passaggio a non privilegiato è a senso unico: cambiare `CONTROL` è a sua volta un'operazione privilegiata. **Attenzione:** il codice non privilegiato che tocca SysTick, NVIC o SCB causa un **BusFault**. **Esempio:** ogni handler gira in Handler mode (privilegiato); al ritorno la CPU riparte dopo la `svc`, di nuovo non privilegiata.
L'app chiede, il kernel fa	L'app non chiama mai un driver e non tocca mai un registro: ogni I/O attraversa il confine con una system call, come `printf` → `write` in Linux.	**Esempio:** `app_main` → `printf` → `write_all` → `sys_write` → `svc #1` → **confine** → `SVC_Handler` → `svc_dispatch` → `uart_write` → UART0 (`0x40004000`). **Regola:** sopra il confine: Thread mode non privilegiato. Sotto: Handler mode privilegiato.
Protezione logica e fisica	Senza MPU l'hardware blocca solo lo System Control Space (protezione *fisica*). I registri UART restano raggiungibili: tenerli lontani è una regola *logica* del design.	**Attenzione:** `irq_save()` usa `cpsid i`, che in modalità non privilegiata viene ignorata in silenzio: il driver crederebbe di essere protetto. **Regola:** il driver UART deve girare solo nel kernel privilegiato.

Concetto	In parole semplici	Esempio, errore tipico, regola
2. ABI, servizi e stub		
ABI *Application Binary Interface*	Il contratto a basso livello tra compilatore e hardware. Un'API parla di nomi C, l'ABI di registri e istruzioni, perché dopo la compilazione i nomi spariscono.	**Regola:** numero del servizio dentro `svc #n`; argomenti in `r0`, `r1`, `r2` (AAPCS); risultato in `r0`. **Esempio:** una syscall sembra una funzione C, ma scatta una `svc` invece di una `bl`.
Servizi	Le costanti `SYS_WRITE`, `SYS_READ`, `SYS_UPTIME` stanno in `lib/syscalls.h`, condiviso da utente e kernel.	**Esempio:** 1 `sys_write(buf, len)`: byte accettati (da 0 a `len`). 2 `sys_read()`: un byte o `-1`. 3 `sys_uptime()`: ms dal boot. Numero 0 o ≥ 4: `-1`. **Regola:** nessun servizio aspetta: `sys_write` prende ciò che entra nel ring buffer e torna subito.
Stub	Funzione assembly di due istruzioni (`svc #n` e `bx lr`) in `lib/syscalls.s`. Sospende l'app e chiama il kernel, senza altra logica.	**Regola:** il numero dopo `svc` è un **letterale** (1, 2, 3), perché è codificato nell'opcode. **Attenzione:** l'assembly non legge le macro C: i numeri vanno tenuti allineati a mano con `lib/syscalls.h`.
Exception Frame e viaggio di una syscall	L'hardware salva 8 registri sullo stack. Il kernel li vede come array `uint32_t *frame`: `frame[0]` è il vecchio `r0`, `frame[1]` il vecchio `r1`.	**Esempio:** (1) C mette gli argomenti in `r0`, `r1`; (2) `svc #1` e stacking; (3) il kernel legge il frame e scrive il risultato in `frame[0]`; (4) l'hardware ripristina i registri, quindi `r0` = risposta; (5) `bx lr`. **Regola:** il risultato va in `frame[0]`, perché al ritorno l'hardware ripopola `r0` da lì.
3. Entrata nel kernel		
`SVC_Handler` in assembly	Deve trovare l'indirizzo del frame, cioè lo stack pointer all'ingresso, e passarlo al C. Due istruzioni: `mrs r0, msp` e `b svc_dispatch`.	**Attenzione:** in C il compilatore aggiunge un prologo (es. `push {r7, lr}`) che sposta `sp` e cambia con `-O0` / `-O2`: si perde il frame. **Regola:** `b` e non `bl`: `bl` sovrascrive `lr` e distrugge `EXC_RETURN` (`0xFFFFFFF9`).

Concetto	In parole semplici	Esempio, errore tipico, regola
Estrarre numero e argomenti	Il numero sta dentro l'istruzione `svc`, quindi il kernel lo rilegge dal codice. In Thumb è a 16 bit: byte alto `0xDF` (opcode), byte basso `n`.	**Esempio:** `svc #1` = `0xDF01`, in memoria (little-endian) `01 DF`. **Regola:** frame = `r0-r3, r12, lr, pc, xPSR`. Il `PC` salvato (`frame[6]`) punta dopo la `svc`: `0xDF` sta a `PC - 1`, `n` a `PC - 2`. Argomenti da `frame[0..2]`.
Dispatcher	Tabella di puntatori a funzione: l'indice è il numero del servizio, l'indice 0 resta vuoto (`NULL`). `svc_dispatch` legge `n` da `PC - 2`.	**Regola:** se `n >= TABLE_SIZE` o `table[n] == 0`: `frame[0] = -1`. Altrimenti `frame[0] = table[n](frame[0], frame[1], frame[2])`. **Esempio:** numero 0 o 99 → `-1`.
4. Dove funziona svc e perché il kernel non aspetta		
Priorità e HardFault	SVCall ha priorità `0x80`. Un'eccezione interrompe solo codice meno urgente (valore più alto = meno urgente), e una `svc` non si può rimandare.	**Esempio:** accettata da Thread mode e da `SysTick_Handler` (`0xE0`) o `UART0TX_Handler` (`0xC0`), dove si annida. **Attenzione:** HardFault se eseguita dentro `SVC_Handler` e funzioni che chiama (`0x80`), in un fault handler (`0x00`) o con `PRIMASK = 1`. **Regola:** per questo il kernel tiene il suo `kprintf` a polling.
Il kernel non aspetta mai	Dentro un handler non si attende. Se `sys_write_impl` ciclasse in SVCall (`0x80`) in attesa del buffer, l'interrupt TX (`0xC0`, meno urgente) non partirebbe mai: deadlock. Anche SysTick (`0xE0`) si bloccherebbe e il clock si fermerebbe.	**Regola:** il kernel accetta quanto entra e torna subito. L'attesa (`wfi`) si fa in Thread mode, dove ogni interrupt può girare.
5. Libreria utente		
`printf` vs `kprintf`	`printf` utente: `vformat` formatta in un buffer locale da 64 byte sullo stack, poi `sys_write` lo invia. `kprintf` kernel: usa direttamente il driver in polling.	**Esempio:** `printf`: non privilegiato, passa da `svc` e ring buffer, attende con `wfi`. `kprintf`: privilegiato, busy-wait. **Attenzione:** `printf` dentro un handler → HardFault. **Regola:** `kprintf` funziona anche in HardFault, perché non usa `svc`.

Concetto	In parole semplici	Esempio, errore tipico, regola
`write_all`	Cicla finché tutti i byte sono inviati, gestendo le scritture parziali di `sys_write`.	**Regola:** se `sys_write` ritorna 0, il buffer è pieno: `wfi` invece di busy-wait. La CPU dorme finché un interrupt (UART TX che libera spazio) la risveglia.
`sleep_ms`	Salva `start = sys_uptime()` e fa `wfi` finché `sys_uptime() - start < ms`.	**Regola:** sottrazione unsigned: regge il wraparound (∼49,7 giorni). **Esempio:** SysTick scatta ogni 10 ms, quindi ogni `wfi` si risveglia almeno così spesso e il kernel non viene mai bloccato.
6. Avvio e applicazione		
Boot di `main`	`main` è l'inizializzatore del kernel: gira privilegiato una volta sola, poi lascia il posto all'app.	**Esempio:** 1) `uart_init()`; 2) `kprintf` del banner; 3) `systick_init()` (tick 10 ms); 4) privilege drop: `CONTROL |= 1` + `isb`; 5) `app_main()`, che non ritorna. **Regola:** l'`isb` scarta le istruzioni già prefetchate sotto le vecchie regole.
`app_main`	App non privilegiata: usa solo `lib/syscalls.h` e `lib/printf.h`. Ogni 1000 ms stampa l'uptime, poi controlla `sys_read()` (`-1` = nessun tasto), fa l'eco e chiama `sleep_ms(10)`.	**Attenzione:** contare le iterazioni accumulerebbe deriva (il tempo speso in `printf`). **Regola:** confrontare `now - last >= 1000` e fare `last += 1000`. **Esempio:** interrupt e SVCall girano privilegiati anche se l'app non lo è: l'hardware alza il privilegio all'ingresso e lo riabbassa al ritorno.
7. Esperimenti e GDB		
Esperimento 1: `printf` nel kernel	Si aggiunge `printf("uptime called\n")` in `sys_uptime_impl`. Invece di una stampa ogni 10 ms si ottiene un HardFault.	**Esempio:** output `HardFault: HFSR=0x40000000` (bit 30 `FORCED` = fault escalato). **Attenzione:** la `svc` parte dentro SVCall (`0x80`), che non è più urgente di sé stessa: non si può rimandare, quindi la CPU la escala. **Regola:** il report si stampa lo stesso perché `HardFault_Handler` usa `kprintf`.

Concetto	In parole semplici	Esempio, errore tipico, regola
Esperimento 2: kernel che aspetta	`sys_write_impl` cicla finché tutto è accettato e `app_main` scrive 130 byte (buffer da 64). Output: un solo `0`, poi silenzio.	**Esempio:** GDB: `p $xpsr & 0x1ff` = 11 (SVCall); `bt`: `uart_write` ← `sys_write_impl` ← `svc_dispatch` ← `<signal handler called>` ← `sys_write` ← `app_main`. **Attenzione:** la prima `uart_write` riempie il buffer e innesca la UART (lo `0`). Solo `UART0TX_Handler` (`0xC0`) potrebbe svuotarlo, ma la CPU è in SVCall (`0x80`): TX pendente per sempre, SysTick fermo. **Regola:** soluzione: il kernel non aspetta, ad attendere è `write_all` in Thread mode.
Privilegio e livelli in GDB	Si verifica la separazione dei livelli con `$control` e il backtrace.	**Esempio:** `p/x $control`: `0x0` in `main`, `0x1` in `app_main`. Dentro `sys_write_impl` è ancora `0x1`: la CPU è privilegiata perché è in Handler mode, non perché `CONTROL` sia cambiato. **Regola:** i 9 bit bassi di `xPSR` sono il numero dell'eccezione (`0xb` = 11 = SVCall). **Regola:** nel backtrace `<signal handler called>` marca l'ingresso in un'eccezione: sopra gira il kernel privilegiato, sotto l'app non privilegiata.